

The Biggest Misconception of Computer Science

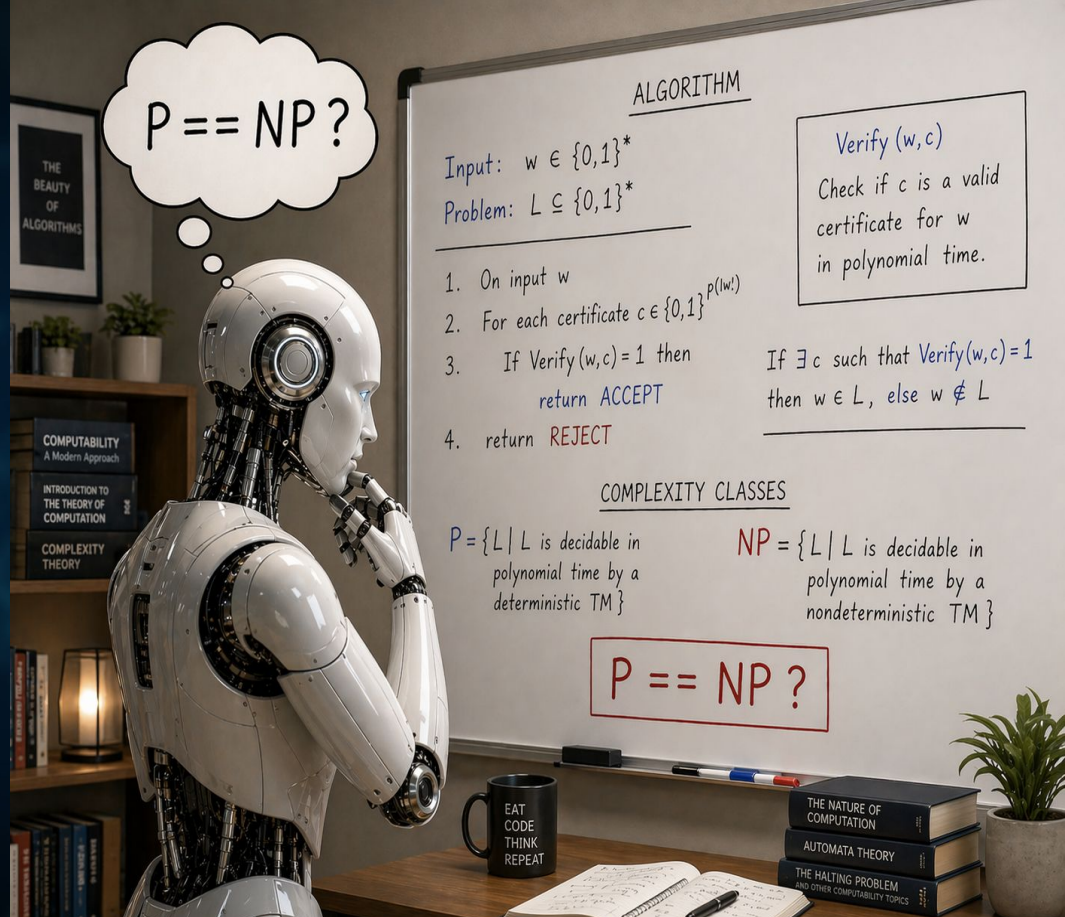
ALEX DATHSKOVSKY



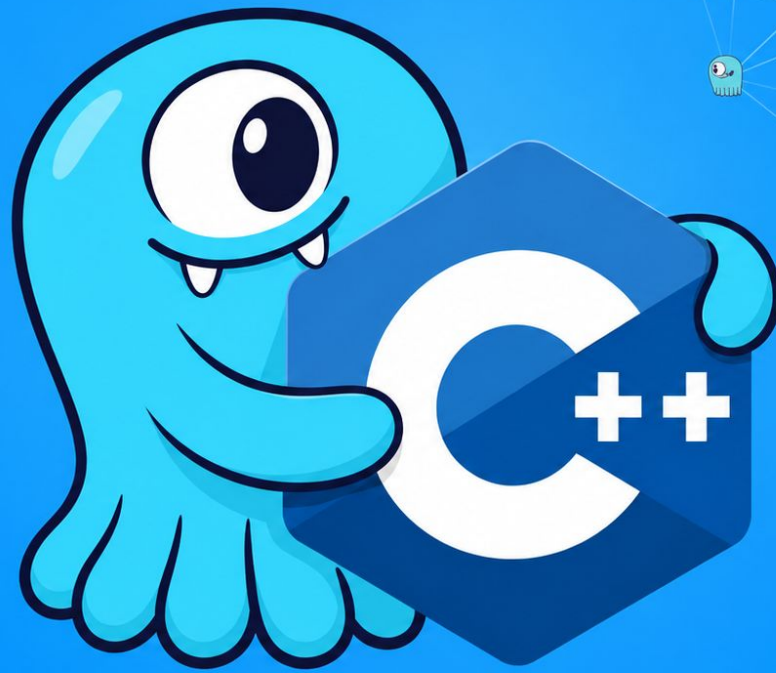
20
26



The Greatest Misconception in Computer Science



About Me:



CONNECT ON
LinkedIn
Alex Dathskovsky



SCAN TO CONNECT

[linkedin.com/in/alexdatahskovsky](https://www.linkedin.com/in/alexdatahskovsky)

VISIT MY WEBSITE

CPPNEXT



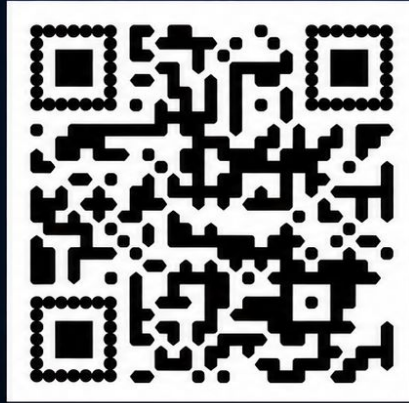
SCAN TO EXPLORE

www.cppnext.com

WATCH ON



@cppnext-alexd



SCAN TO SUBSCRIBE

youtube.com/@cppnext-alexd

LISTEN ON
SUNO
@calebxyz376



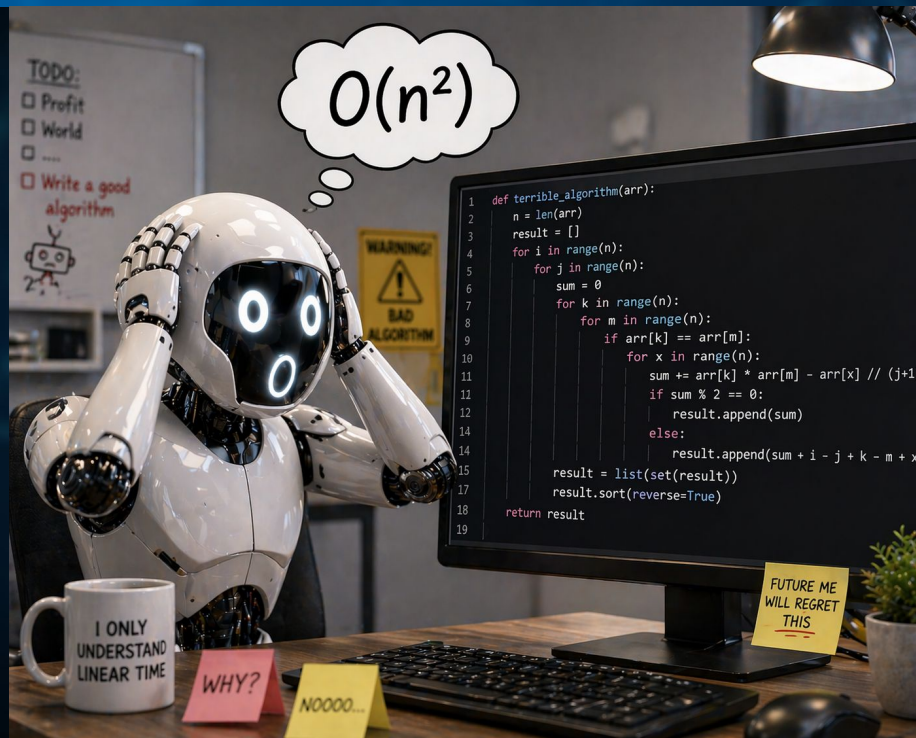
SCAN TO LISTEN

suno.com/@calebxyz376

Let's Start With a Story

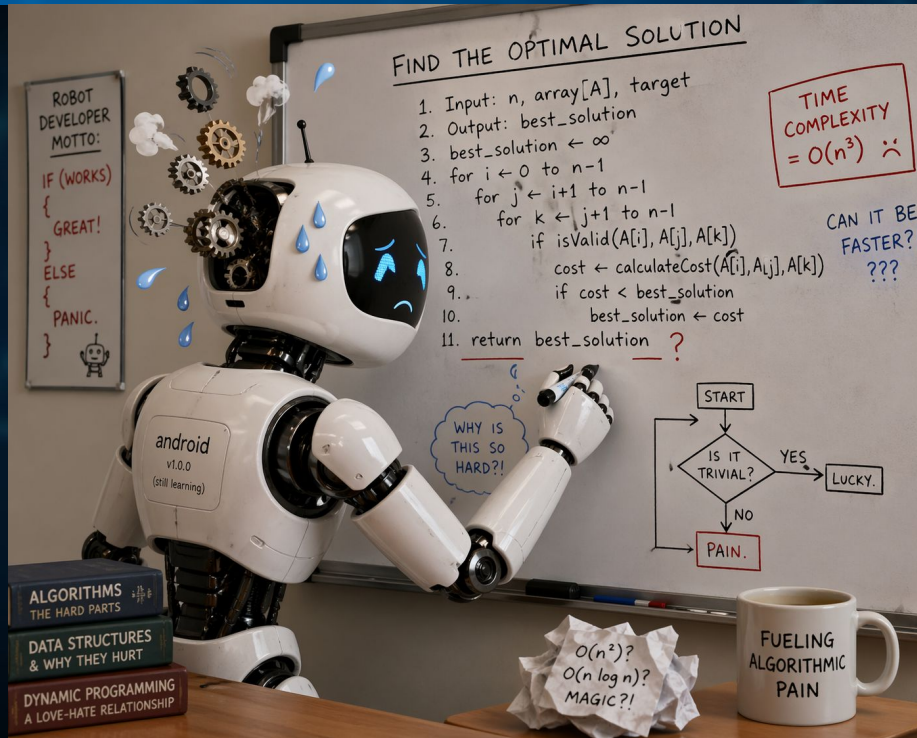
Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

Story Time



Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

Story Time

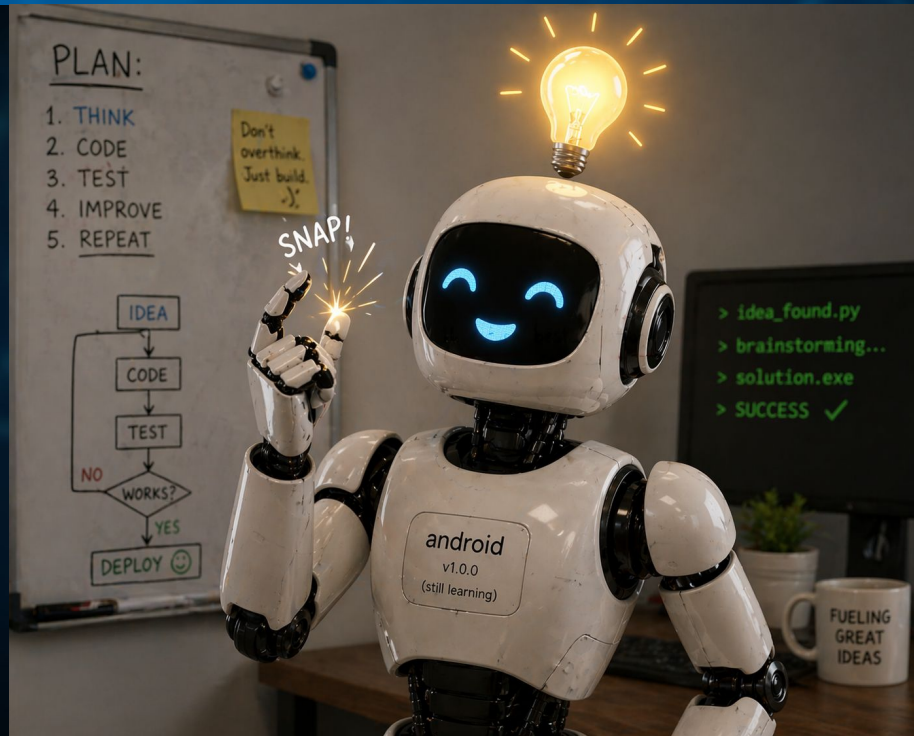


[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky



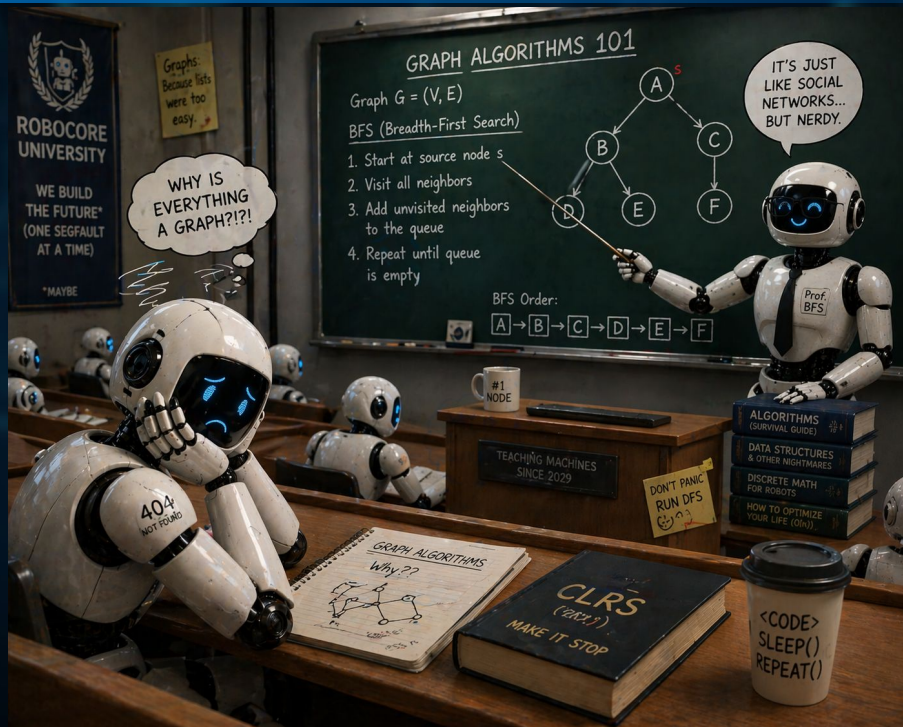
[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Story Time



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Story Time



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Complexity

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

Complexity

- ❑ $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$
- ❑ This is bad $O(n^2)$
- ❑ And if you wrote $O(n^3)$... straight to jail with you!

```
int linear(int n) {  
    int sum = 0;  
  
    for (int i = 0; i < n; ++i) {  
        sum += i;  
    }  
  
    return sum;  
}
```

```
int logarithmic(int n) {  
    int sum = 0;  
  
    for (int i = 1; i < n; i *= 2) {  
        sum += i;  
    }  
  
    return sum;  
}
```

Complexity

```
int quadratic(int n) {  
    int sum = 0;  
  
    for (int i = 0; i < n; ++i) {  
        for (int j = 0; j < n; ++j) {  
            sum += i * j;  
        }  
    }  
  
    return sum;  
}
```

[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

Complexity

```
int combined(int n) {  
    int sum = 0;  
  
    // O(n)  
    for (int i = 0; i < n; ++i) {  
        sum += i;  
    }  
}
```

Total Complexity: $O(N) + O(N^2) + O(\log N)$

Your algorithm is only as good as its worst part.

$O(N) + O(N^2) + O(\log N) \Rightarrow O(N^2)$

```
    // O(log n)  
    for (int i = 1; i < n; i *= 2) {  
        sum += i;  
    }  
  
    return sum;  
}
```

```
// O(n)
int sum = 0;
for (int i = 0; i < n; ++i)
    sum += arr[i];
```



```
// O(log n)
for (int i = 1; i < n; i *= 2)
    do_something_heavy();
```

Data Structure - Complexity

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

Data Structure - Complexity

Data Structure	Insert	Delete	Find / Search
Array	$O(1)$	$O(1)$	$O(N) / O(\log N)$
Linked List	$O(N)$	$O(N)$	$O(N)$
Binary Tree	$O(\log N)^*$	$O(\log N)^*$	$O(\log N)^*$
Heap	$O(\log N)$	$O(\log N)$	$O(N)$
Stack	$O(1)$	$O(1)$	$O(N)$

Problem: Group Subtraction

Write a function that:

- ❑ Receives two vectors of numbers: A and B
- ❑ Treats them as two groups
- ❑ Returns a new vector where:
 - ❑ All elements from A that appear in B are removed
 - ❑ Order of A must be preserved

Example

Input:

- A = [1, 2, 3, 4, 5, 3, 2, 6]
- B = [2, 3]

Output:

- Result = [1, 4, 5, 6]

Naive Solution

```
3  std::vector<int> group_subtract(const std::vector<int>& A,  
4                                const std::vector<int>& B){  
5      std::vector<int> result;  
6      for (const auto& a : A) {  
7          bool found = false;  
8          for (const auto& b : B) {  
9              if (a == b) {  
10                 found = true;  
11                 break;  
12             }  
13         }  
14         if (!found) {  
15             result.push_back(a);  
16         }  
17     }  
18     return result;  
19 }
```

[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Linear Solution

```
23  std::vector<int> group_subtract_linear(const std::vector<int>& A,  
24  |   |   |   |   |   |   |   const std::vector<int>& B)  
25  {  
26      std::vector<int> result;  
27  
28      // Preprocess B into a hash set (O(N))  
29      std::unordered_set<int> lookup(B.begin(), B.end());  
30  
31      // Single pass over A  
32      for (const auto& a : A) {  
33          // O(1) lookup  
34          if (lookup.find(a) == lookup.end()) {  
35              result.push_back(a);  
36          }  
37      }  
38  
39      return result;  
40  }
```

[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alex-dathskovsky

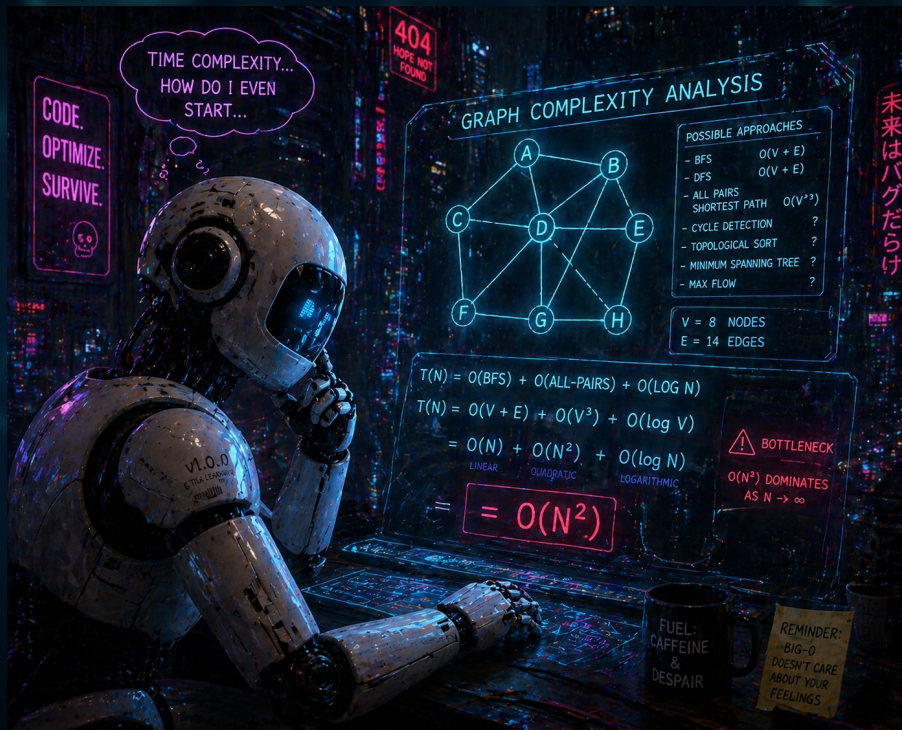
Modern Linear Solution

```
44  std::vector<int> group_subtract_modern(const std::vector<int>& A,  
45  |                                     const std::vector<int>& B){  
46  |     std::unordered_set<int> lookup(B.begin(), B.end());  
47  |  
48  |     return A  
49  |         | std::views::filter([&](int a){  
50  |             |     return !lookup.contains(a);})  
51  |             | std::ranges::to<std::vector>();  
52  | }
```

Algorithms Course Flashback

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

Complexity Just Got Harder



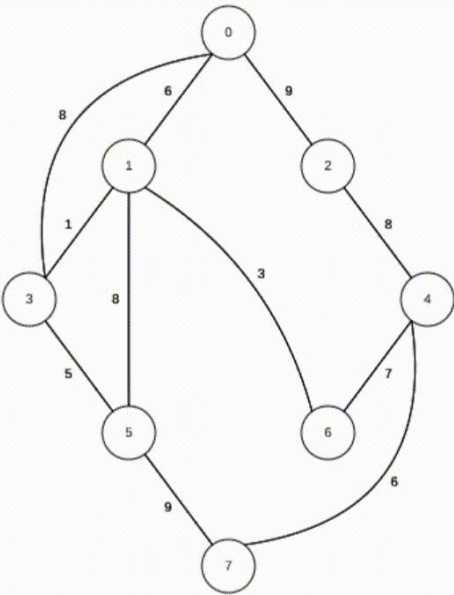
[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdataskovsky

Goal:

Find the shortest path from one source node to all other nodes in a weighted graph.

Dijkstra

Vertex	Known	Cost	Path
0			
1			
2			
3			
4			
5			
6			
7			



Dijkstra

Step	Operation	Data Structure	Cost per Operation	Number of Times	Total Cost
1	Initialize distances	Vector	$O(1)$	V	$O(V)$
2	Extract min node	Min Heap / Priority Queue	$O(\log V)$	V	$O(V \log V)$
3	Relax edges	For each edge ($u \rightarrow v$)	$O(1)$	E	$O(E)$
4	Update priority queue	Decrease-key / push	$O(\log V)$	E	$O(E \log V)$

[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alex-dathskovsky

Algorithms



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

Amortized Complexity

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

Amortized Example std::vector

```
std::vector<int> v;  
v.push_back(x);
```

Amortized Example: Potential Function

Actual cost:

$c_i = 1$ if no reallocation
 $c_i = 1 + \text{old_size}$ if reallocation happens

Potential function:

$$\Phi(v) = 2 \cdot \text{size}(v) - \text{capacity}(v)$$

Amortized cost:

$$\hat{a}_i = c_i + \Phi_i - \Phi_{i-1}$$

Amortized Example: No Reallocation

size: $s \rightarrow s + 1$

capacity: $C \rightarrow C$

$c_i = 1$

$\hat{a}_i = 1 + [2(s + 1) - C] - [2s - C]$

$\hat{a}_i = 1 + 2$

$\hat{a}_i = 3$

Amortized Example: Reallocation

size: $s \rightarrow s + 1$

capacity: $C \rightarrow 2C$

$c_i = s + 1$

$\hat{a}_i = (s + 1) + [2(s + 1) - 2s] - [2s - s]$

$\hat{a}_i = 1 + 2$

$\hat{a}_i = 3$

Amortized Example

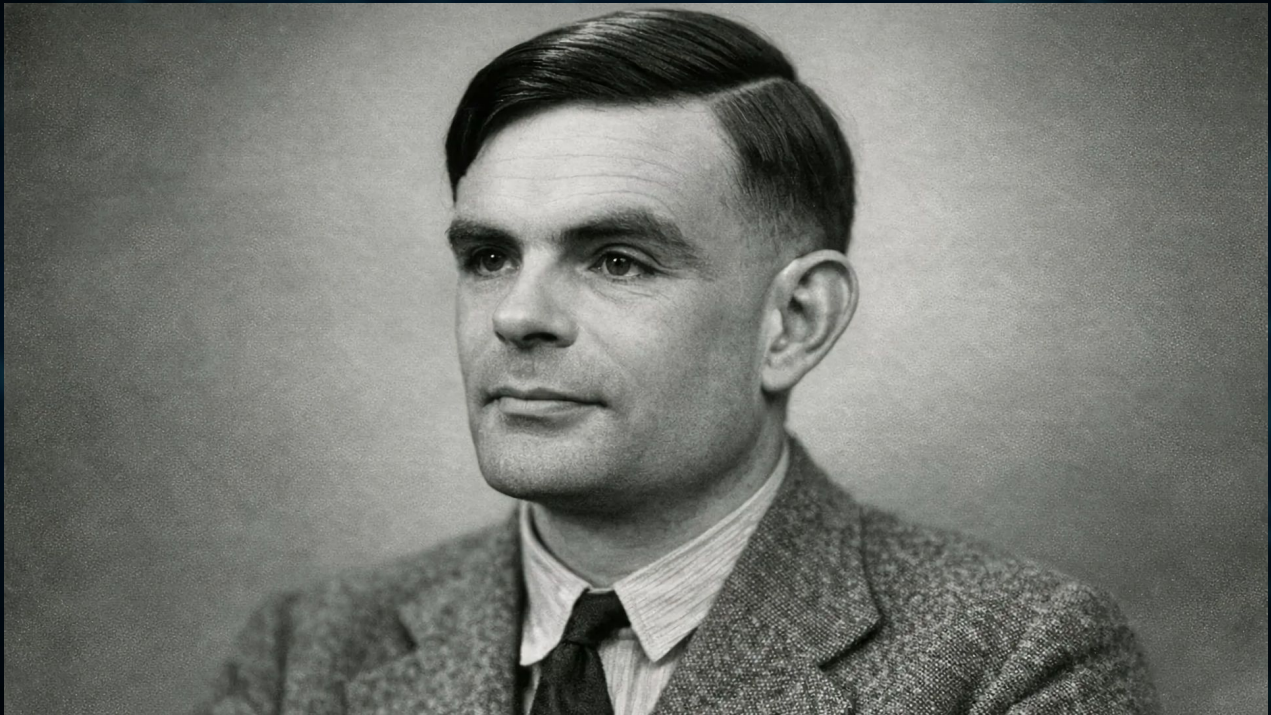


[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Where Did It All Came From?

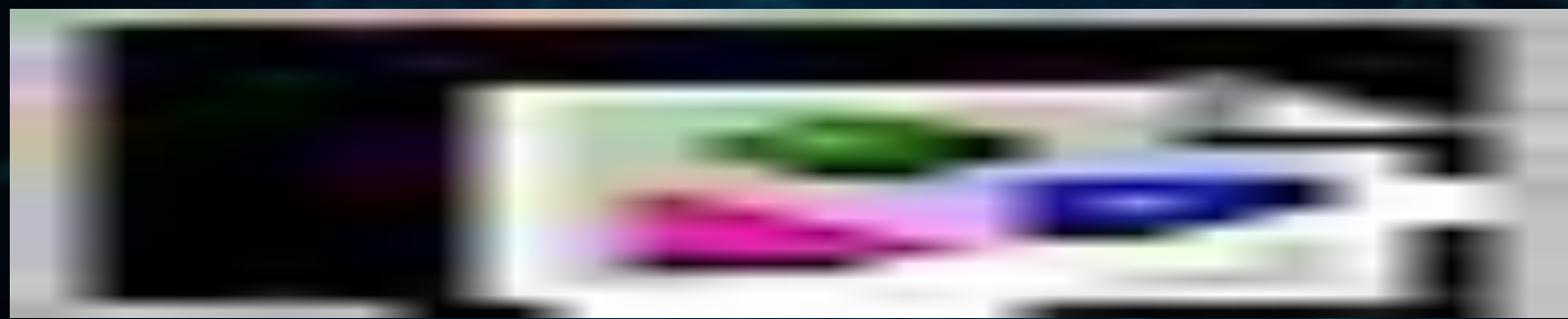
Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

Where Did It All Start?



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Turing Machine



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

Complexity Classes

P → Problems we can solve efficiently

Run Times:

$O(n)$

$O(n \log n)$

$O(n^2)$ (yes, still counts)

Examples:

- Sorting → **P**
- Path finding → **P**

NP → Problems we can verify efficiently

Examples:

- Traveling salesman → NP
- 3-SAT → NP

Sudoku - Example

8	2	7	1	5	4	3	9	6
9	6	5	3	2	7	1	4	8
3	4	1	6	8	9	7	5	2
5	9	3	4	6	8	2	7	1
4	7	2	5	1	3	6	8	9
6	1	8	9	7	2	4	3	5
7	8	6	2	3	5	9	1	4
1	5	4	7	9	6	8	2	3
2	3	9	8	4	1	5	6	7

3-SAT $\leq$ Sudoku

Given a 3-SAT formula:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge \dots$$

we build a Sudoku puzzle such that:

The Sudoku has a valid solution
iff the 3-SAT formula is satisfiable.

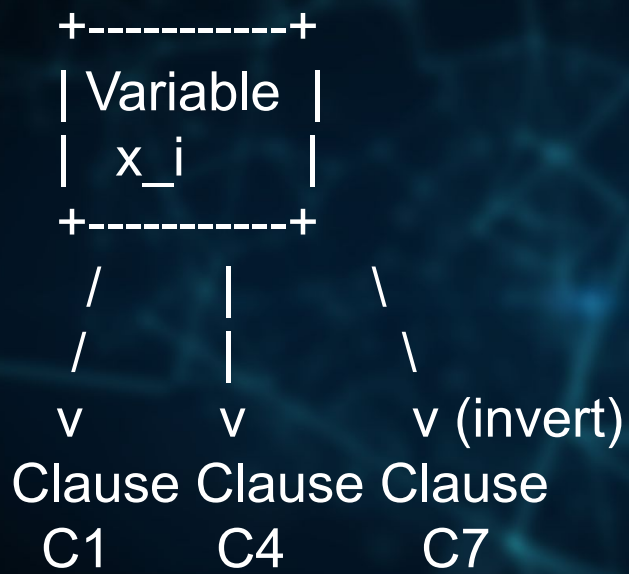
Variables:

Variable x_i

Option 1: choose TRUE

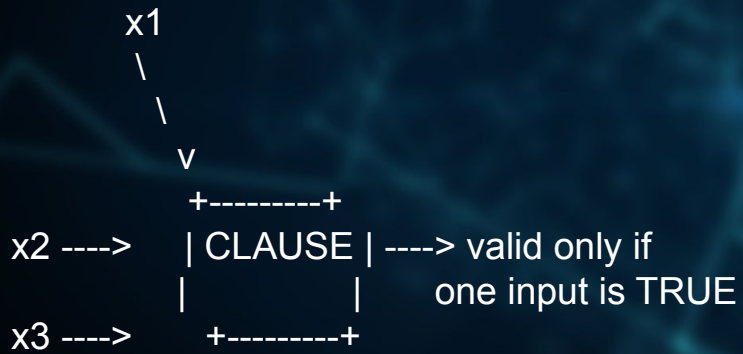
Option 2: choose FALSE

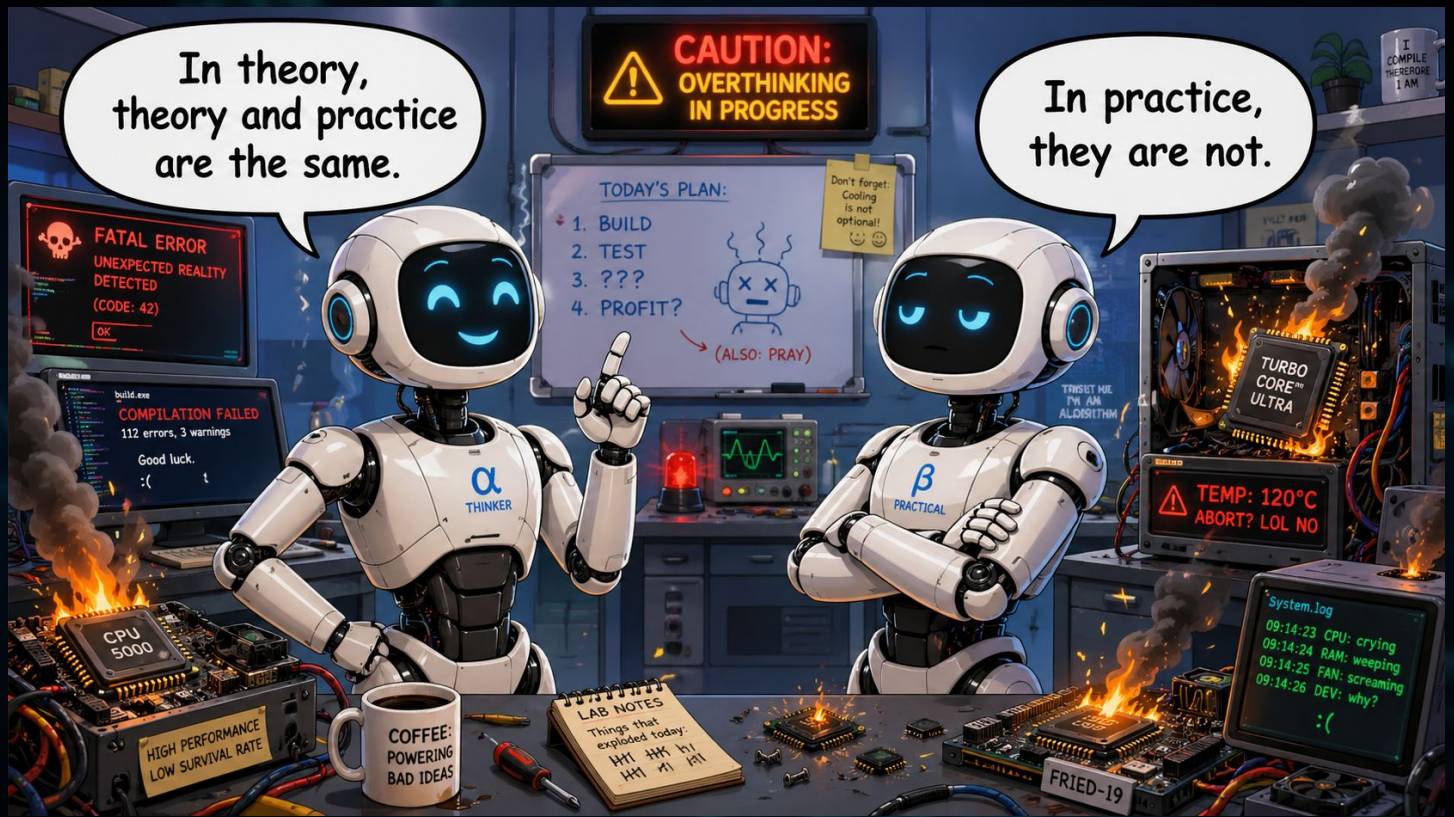
3-SAT $\leq$ Sudoku



$$C_j = (x_1 \vee x_2 \vee x_3)$$

each cell is defined as a clause





CPU Evolution

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

In the Beginning



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

In the Beginning

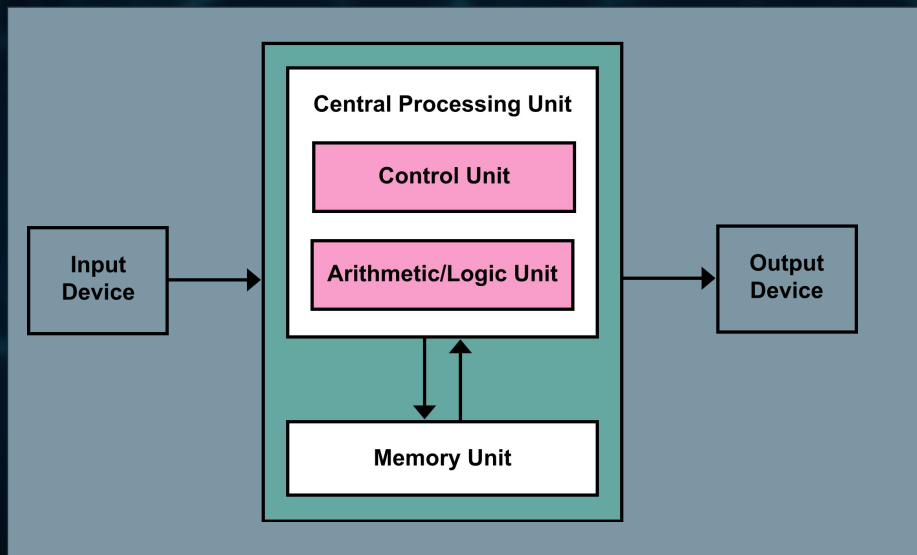


[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Basic Architecture

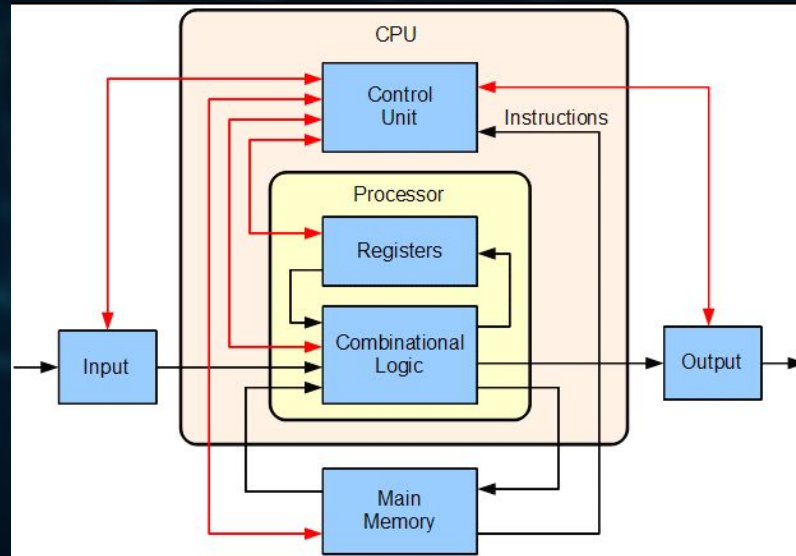
Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

In the Beginning



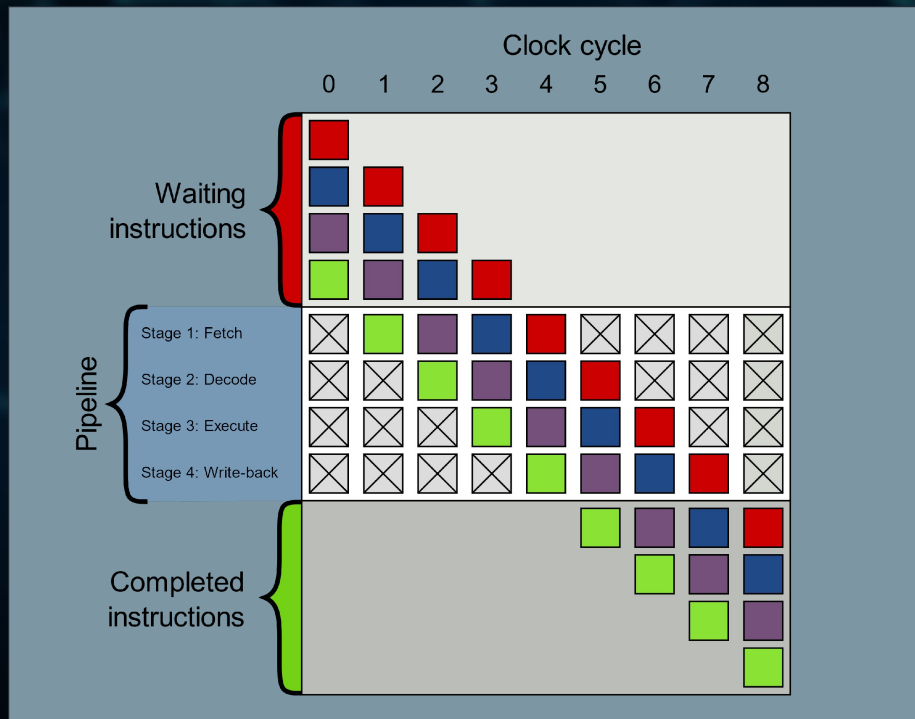
Source: https://en.wikipedia.org/wiki/Von_Neumann_architecture

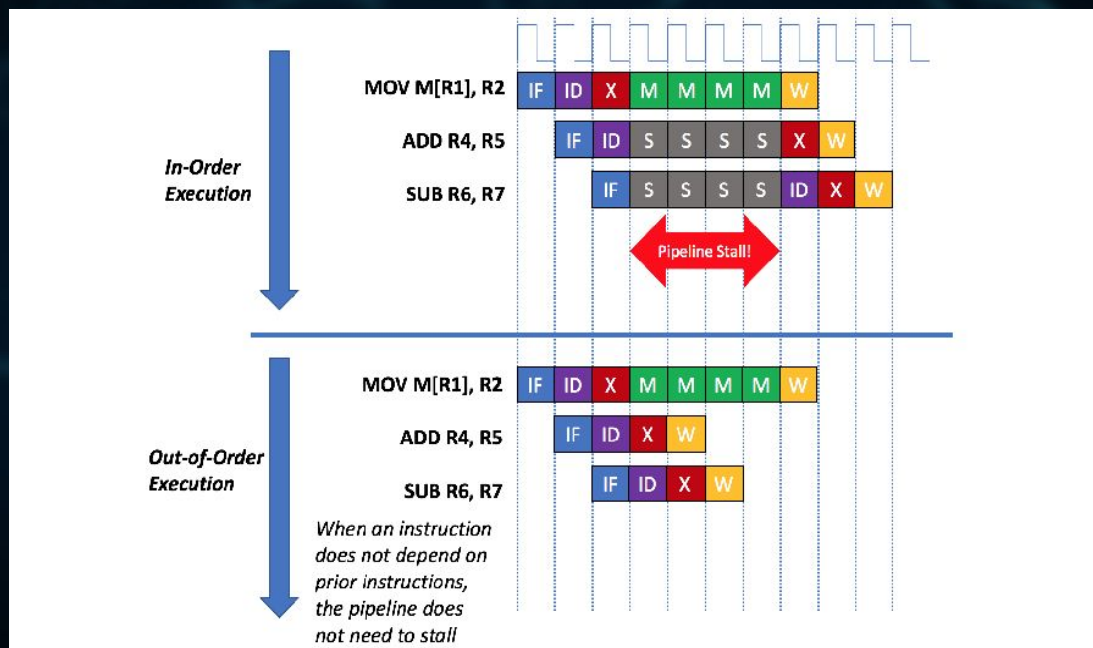
Simple CPU Diagram



Source: https://en.wikipedia.org/wiki/Central_processing_unit

Simple CPU Piped Sequential Execution





Piped Instructions

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

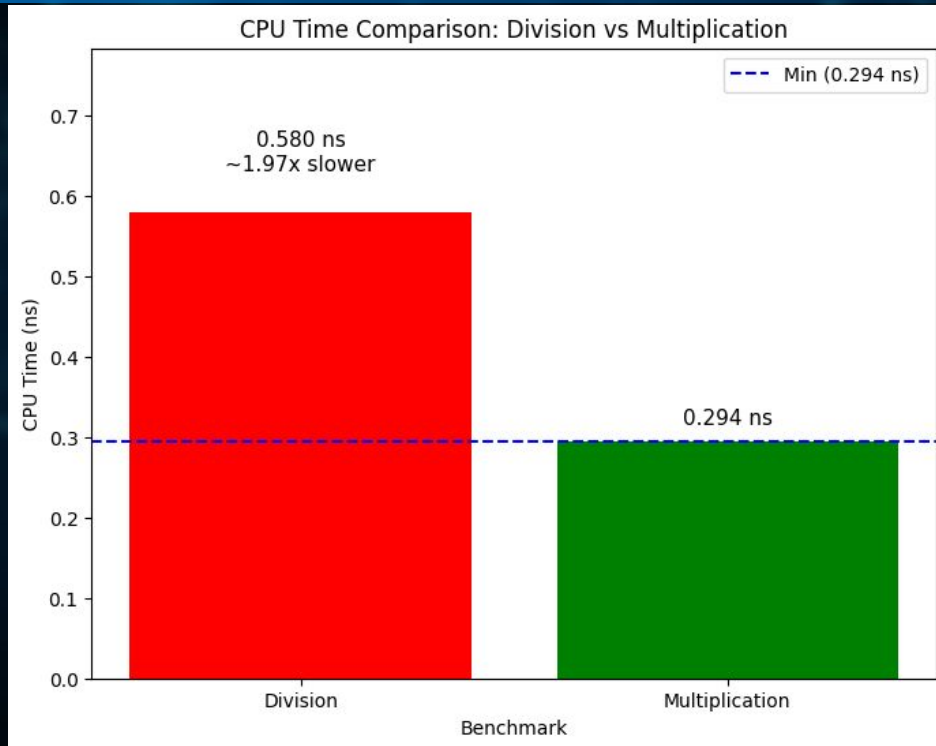
Piped Execution Units

- ADD – 1 cycle (not piped)
- IMUL – 3 cycles
- DIV – at least x20 time slower than imul (depending on the architecture)

Piped Execution Units

```
3   float Percent_With_Division() {  
4       float value = 12345.678;  
5       return value / 100.0f;  
6   }  
7  
8  
9   float Percent_With_Multiplication() {  
10      float value = 12345.678;  
11      return value * 0.01f;  
12  }
```

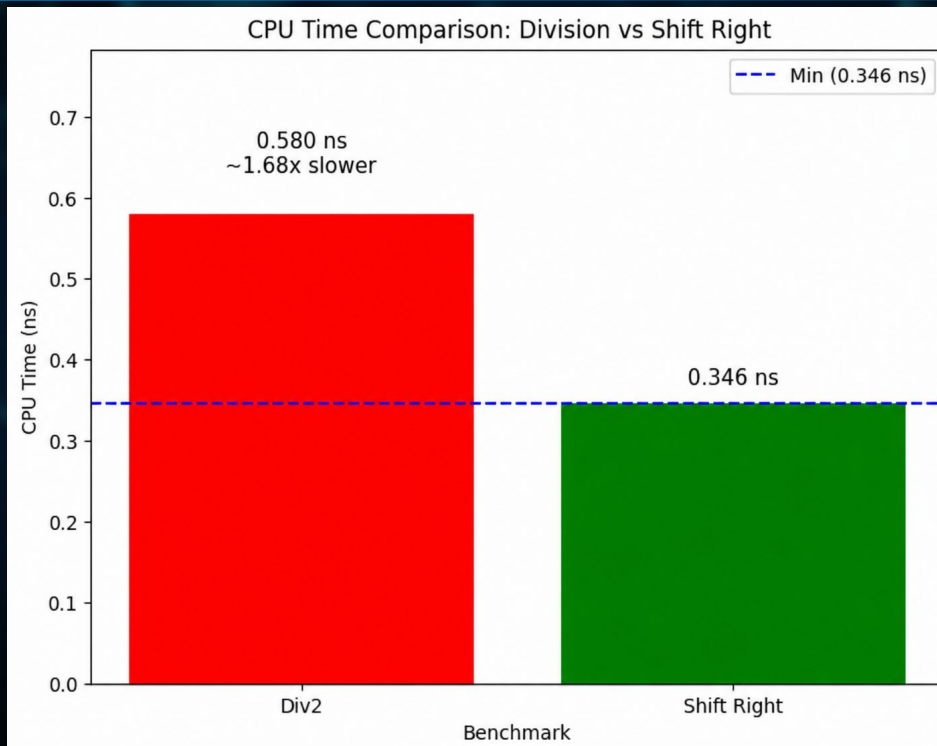
Piped Execution Units



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

```
3  std::uint64_t divide_by_2() {  
4      |    return 12345ULL / 2;  
5  }  
6  
7  std::uint64_t shift_right_by_1() {  
8      |    return 12345ULL >> 1;  
9  }
```

Piped Execution Units



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Piped Execution Units

Cpp
North
2024

think-cell



JETBRAINS

To iNT Gr
To UiNT



**Alex
Dathskovsky**

To Int or to Uint

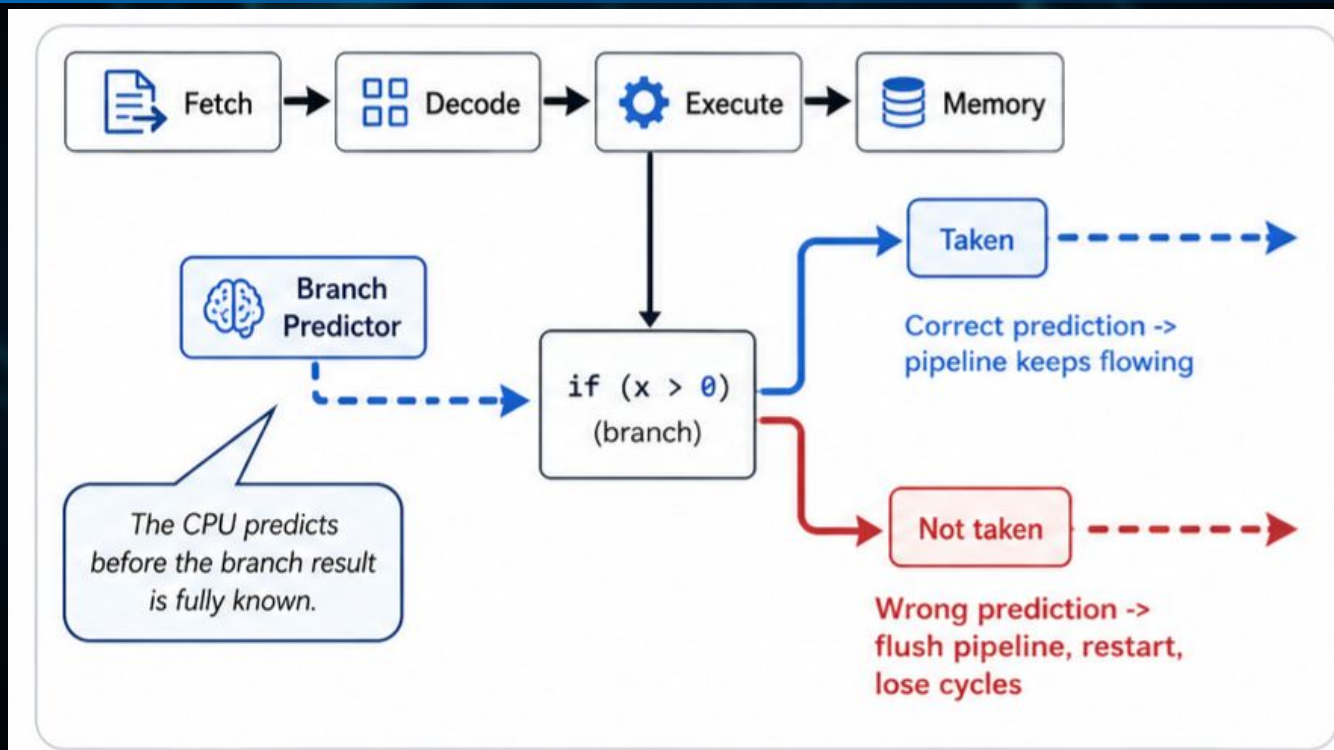


[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

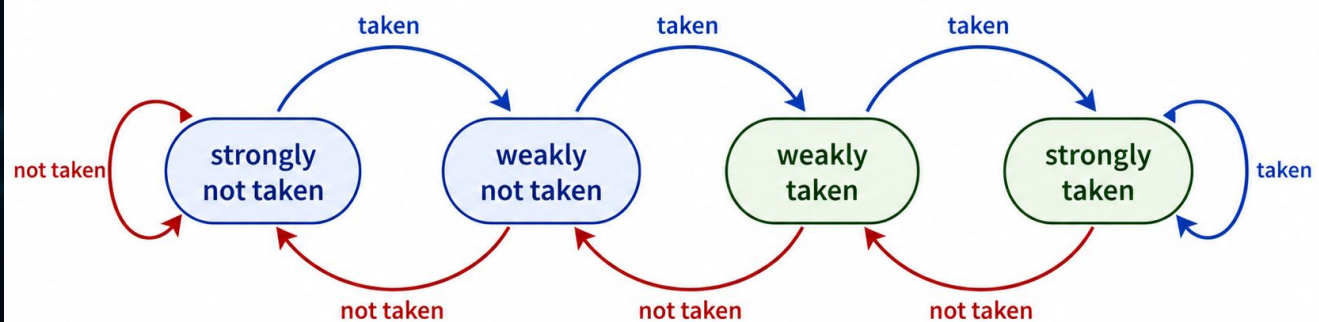
Branch Prediction

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

Branch Prediction



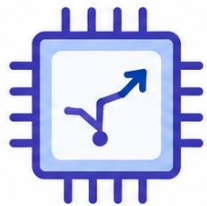
Branch Prediction: Simple Example



Modern Branch Prediction

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

What is TAGE-SC-L?



TAGE-SC-L stands for

TAgged Geometric history length predictor

+ **S**tatistical Corrector

+ **L**oop predictor



A **high-accuracy** branch predictor used in **advanced CPUs**.



Predicts whether a branch will be taken by combining **multiple history lengths**.



Uses **correction logic** to fix hard cases and **loop awareness** to capture repeating behavior.

TAGE-SC-L combines three strengths to deliver highly accurate predictions.



TAGE

Long-history
pattern matching

+



SC

Statistical
correction

+



L

Loop
detection



High-accuracy
branch prediction

Modern Branch Prediction

1

Base predictor

Quick simple prediction



Provides a fast, always-available prediction using basic information (e.g., bias toward taken/not taken).

Used as the default when other components have no opinion.

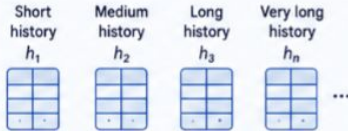
2

TAGE tables

Multi-length history matching



A set of tagged predictor tables, each indexed with a different geometric history length.



Longer histories capture harder patterns.

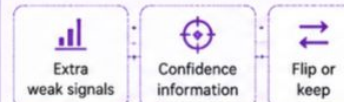
3

Statistical Corrector (SC)

Refine or flip the TAGE result



Uses additional weak signals and confidence information to decide whether to keep the TAGE prediction or flip it.



4

Loop predictor

Detect and predict loops



Detects repeating loop behavior, tracks iteration counts, and predicts the loop-exit (not taken) with high accuracy.

Example

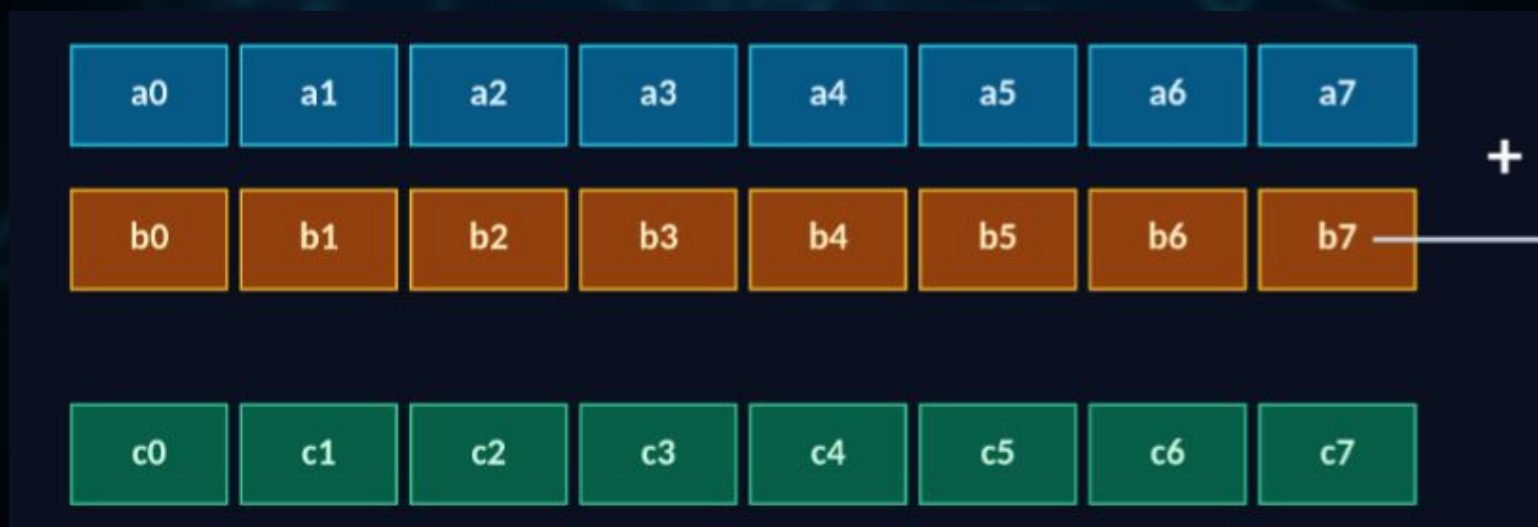


Predicts many TAKENS, then the exit.

SIMD

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

SIMD



COMMON SIMD WIDTHS

SSE (128-bit)

4 × 32-bit floats



AVX (256-bit)

8 × 32-bit floats



AVX-512 (512-bit)

16 × 32-bit floats



NEON (128-bit)

4 × 32-bit floats



The same instruction works on all lanes.

Wider = more data per instruction.

```
8      simd::vec<float, 4> v1([](int i) -> float {
9          return i + 1.5; });
10
11     simd::vec<float, 4> v2([](int i) -> float {
12         return (i + 1.5) * 2.1; });
13
14     auto v3 = v1 + v2;
15
16     float total = simd::reduce(v3);
```

```
movaps    xmm0, XMMWORD PTR [rdi]
addps     xmm0, XMMWORD PTR [rsi]
movaps    xmm1, xmm0
shufps    xmm1, xmm0, 78
addps     xmm1, xmm0
movaps    xmm0, xmm1
shufps    xmm0, xmm1, 85
addss     xmm0, xmm1
ret
```

SIMD and GPGPU



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Branchless Programming

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

Branchless Programming

```
58 void cpu_branching(  
59     const std::vector<int>& input,  
60     std::vector<int>& output)  
61 {  
62     for (std::size_t i = 0; i < input.size(); ++i) {  
63         if (input[i] > 0) {  
64             output[i] = input[i] * 2;  
65         } else {  
66             output[i] = -input[i];  
67         }  
68     }  
69 }
```

Branchless Programming

```
72  __global__ void cuda_branchless(  
73      const int* input,  
74      int* output)  
75  {  
76      int i = blockIdx.x * blockDim.x + threadIdx.x;  
77      int x = input[i];  
78      int positive_result = x * 2;  
79      int negative_result = -x;  
80  
81      output[i] = (x > 0) ? positive_result : negative_result;  
82  }
```

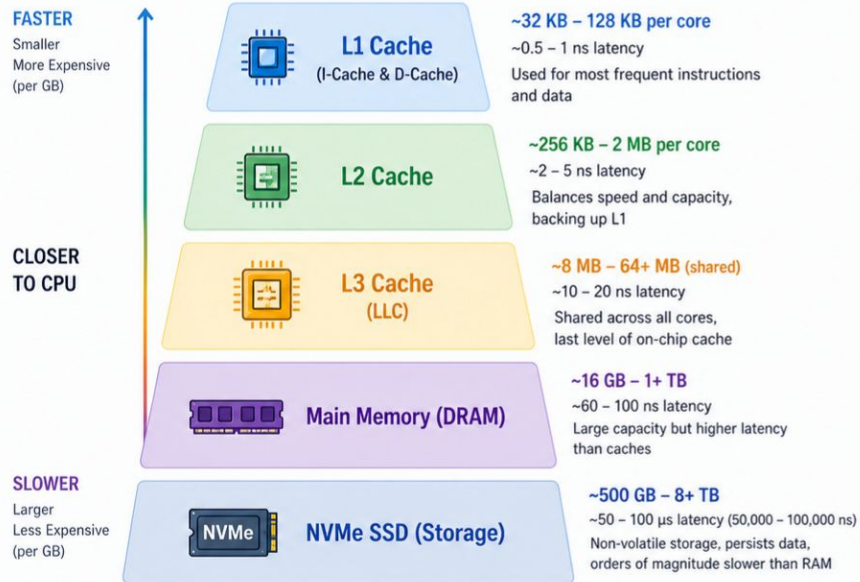
Branchless Programming (ARM V8)

```
lsl      w8, w0, #1  
cmp      w0, #0  
csneg    w0, w8, w0, gt  
ret
```

Cache

Modern CPU Cache Levels & NVMe Storage

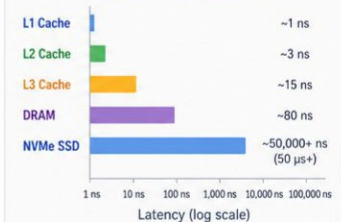
The memory hierarchy brings data closer to the CPU for maximum performance



KEY TAKEAWAYS

- 🎯 Data moves between levels automatically. Caches hold the most recently or frequently used data.
- 🔍 Higher (top) levels are faster but smaller and costlier.
- 🎯 Lower levels are larger but slower.
- 🔍 CPU performance depends on minimizing cache misses and memory stalls.

TYPICAL LATENCY COMPARISON



Example (Cache size = 3)

Access sequence:

0

1

2

0

3

0

4

MRU

7

0

1

2

0

3

0

4

-

7

0

1

2

0

3

0

LRU

-

-

7

0

1

2

2

3

Action

Miss
(0)

Miss
(1)

Miss
(2)

Hit
(0)

Miss
(evict 1)

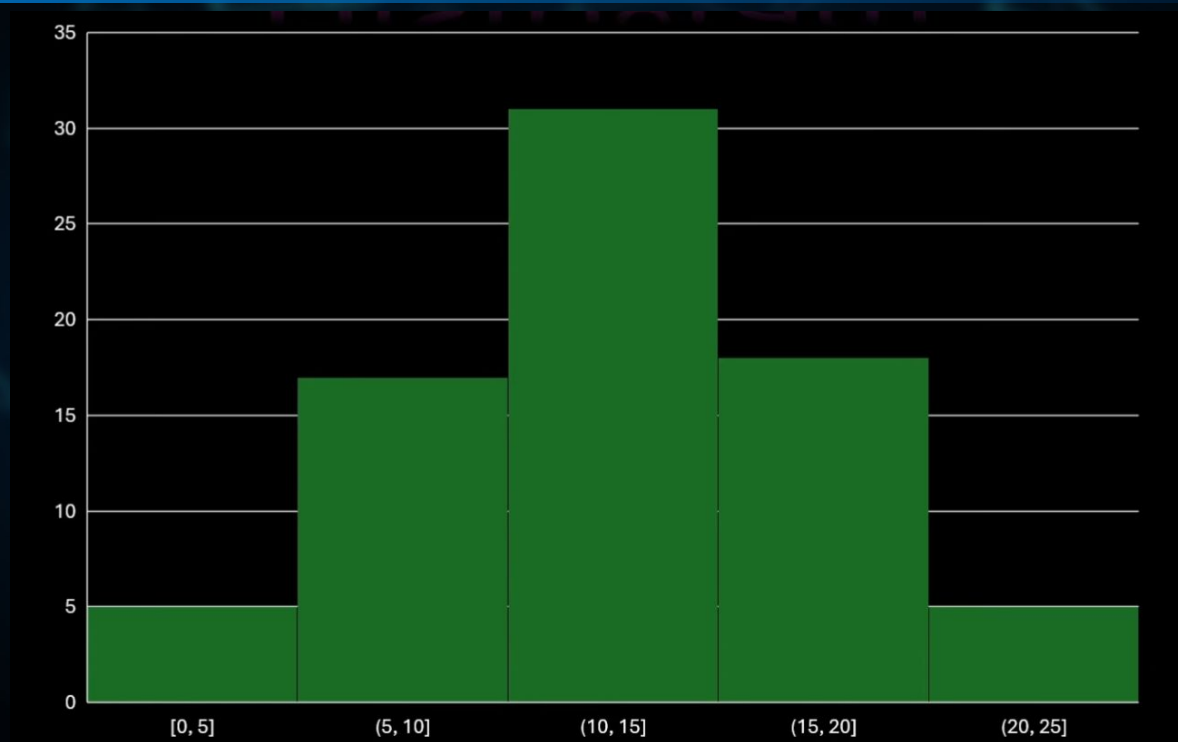
Hit
(0)

Miss
(evict 2)

Histogram Example

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

Histogram



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

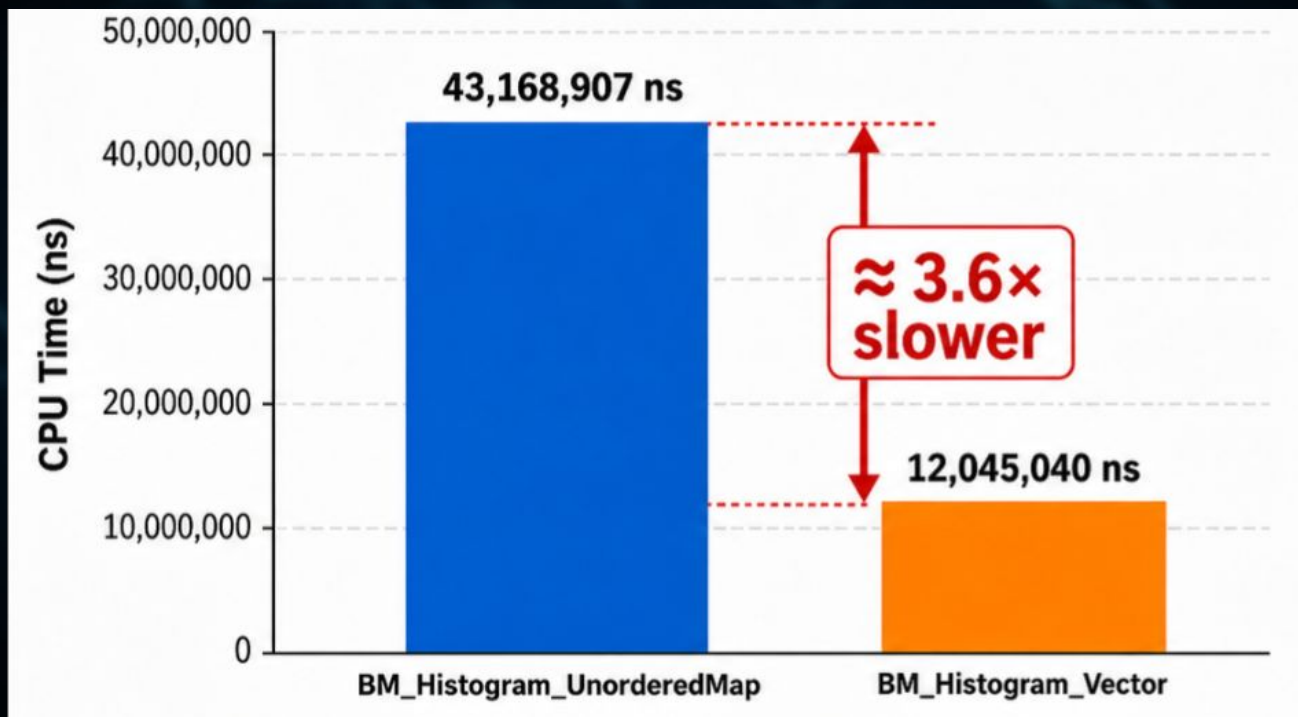
Histogram With std::unordered_map

```
49 static void hist_umap(const std::vector<uint64_t>& data)
50 {
51     std::unordered_map<value_type, count_type> histogram;
52     histogram.reserve(BINS);
53     for (value_type bin = 0; bin < BINS; ++bin) {
54         histogram.emplace(bin, 0);
55     }
56     for (const auto value : data) {
57         ++histogram[value / DIV];
58     }
59 }
```

Histogram With std::vector

```
61 static void hist_vect(const std::vector<uint64_t>& data)
62 {
63     std::vector<count_type> histogram;
64     histogram.resize(BINS);
65     std::ranges::fill(histogram, 0);
66     for (const auto value : data) {
67         ++histogram[value / DIV];
68     }
69 }
```

Benchmark Results

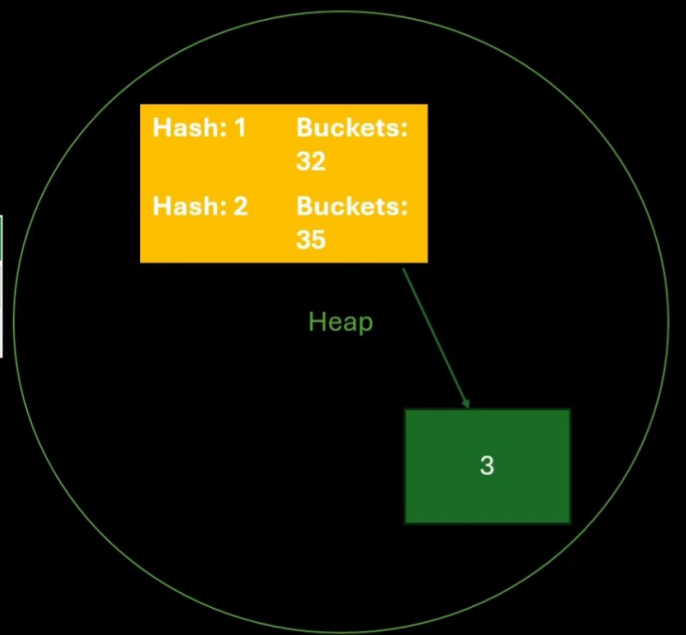


[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Why std::unordered_map Pays More

L1 Data

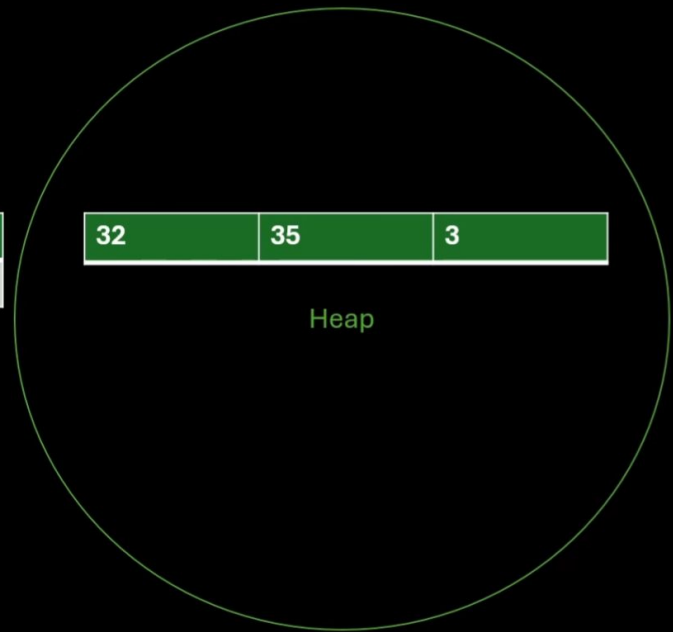
0-7 bytes	8-15 bytes	16-23 bytes	24-31 bytes
32	xx	xx	xx
35	xx	xx	xx



Why std::unordered_map Pays More

L1 Data

0-7 bytes	8-15 bytes	16-23 bytes	24-31 bytes
32	35	3	xx

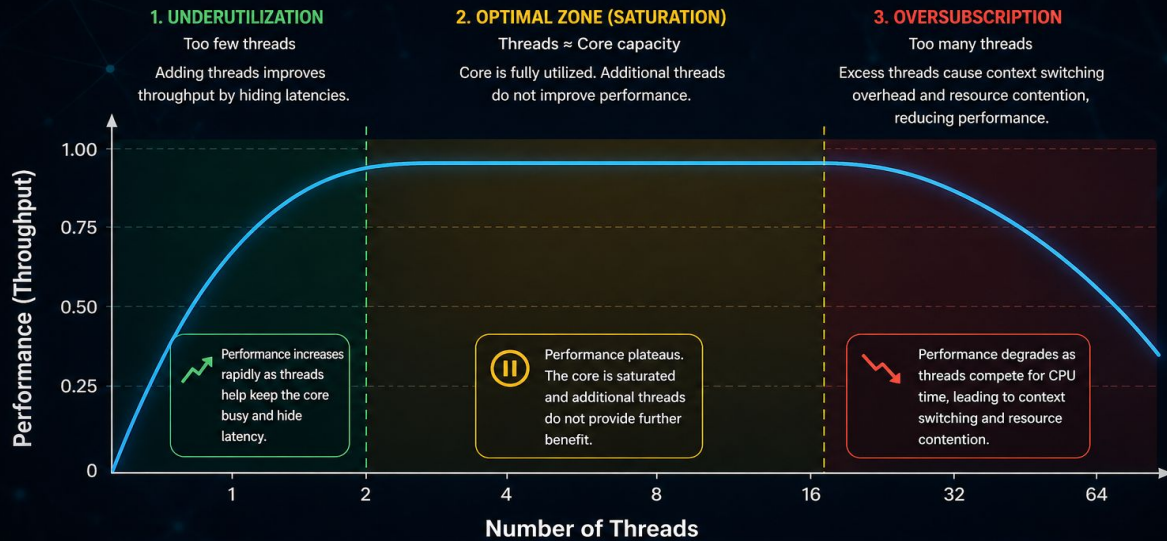


Multi Thread/Core

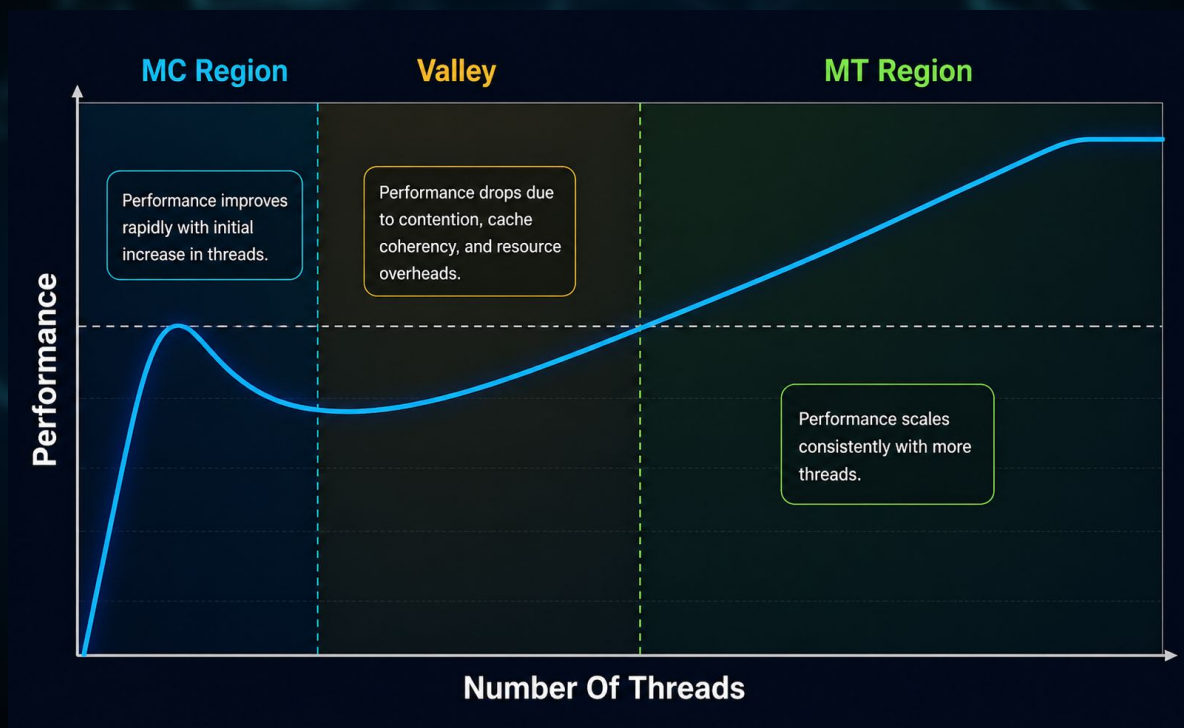
Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

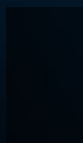
MULTITHREADING ON A SINGLE CORE (NO CACHE)

Effect of Threads on Performance (Throughput)



Multi Core and Caching





Will Throwing More Threads Help

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

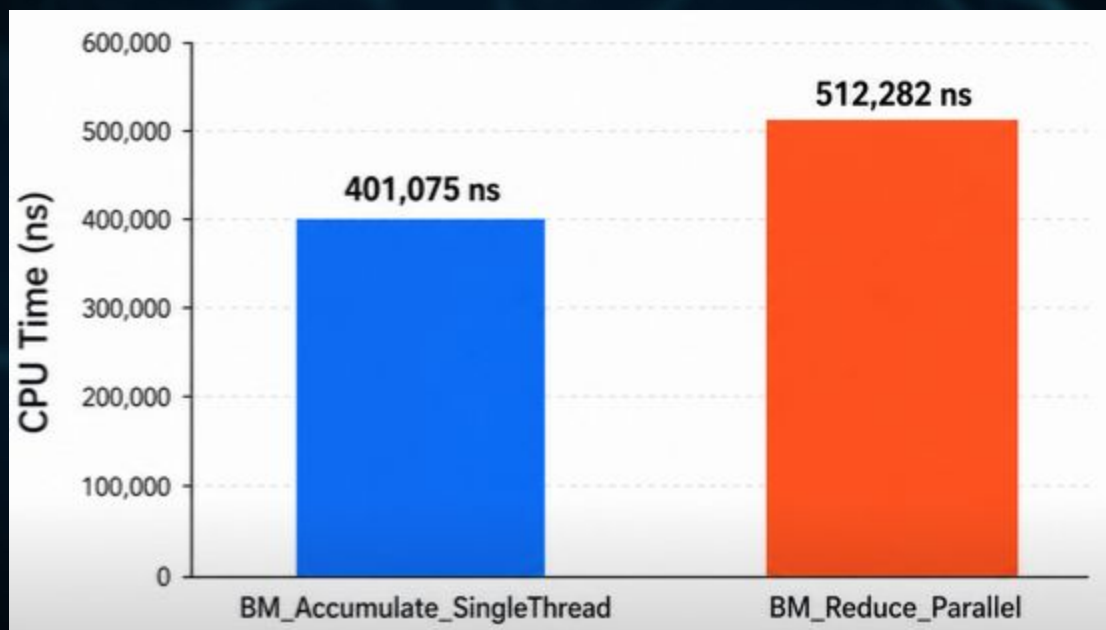
Example

```
auto result = std::accumulate(  
    data.begin(),  
    data.end(),  
    0ull  
);
```

Example

```
auto result = std::reduce(  
    std::execution::par,  
    data.begin(),  
    data.end(),  
    0ull  
);
```

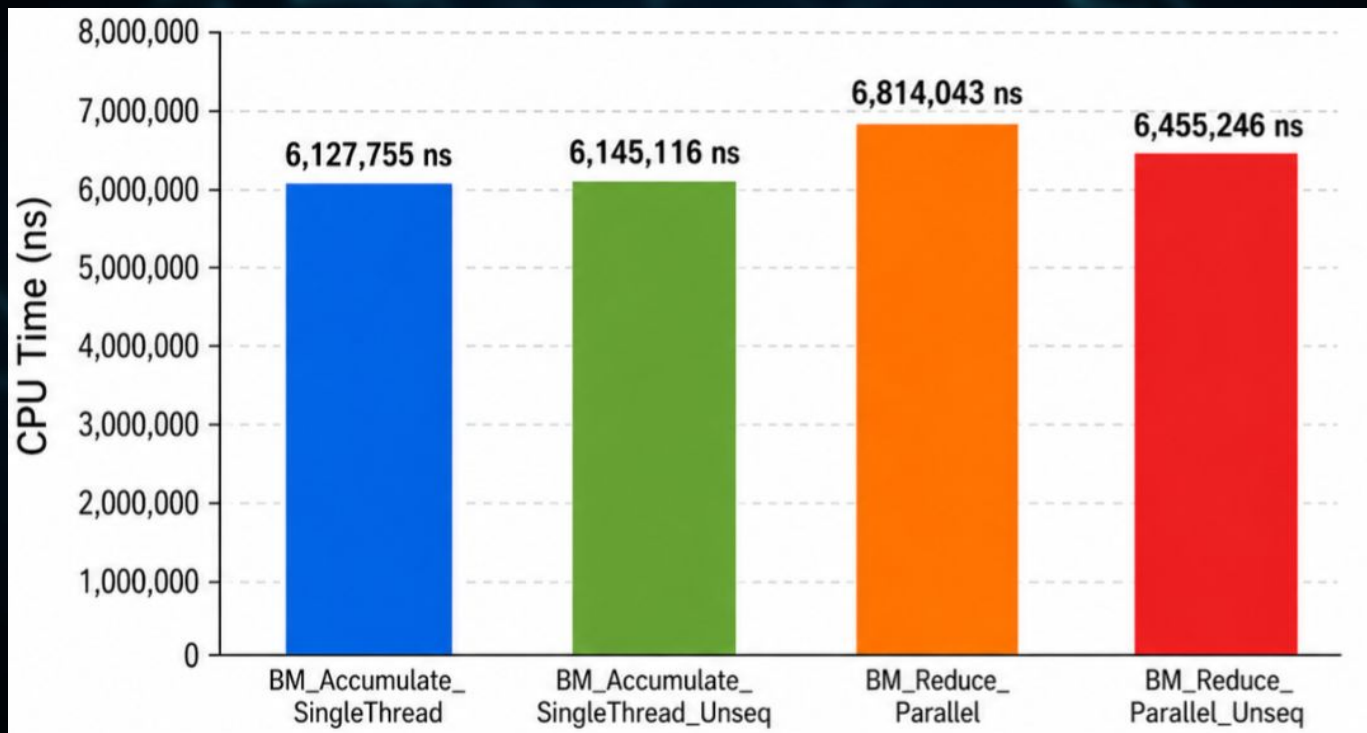
Example



Example - unseq



Example - unseq

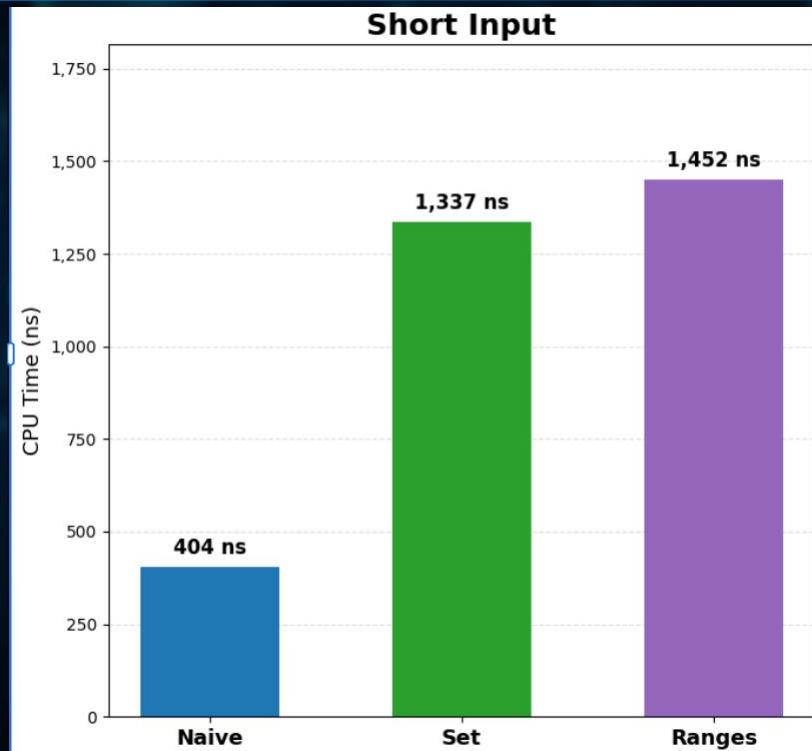


[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Benchmarking The Group Subtraction

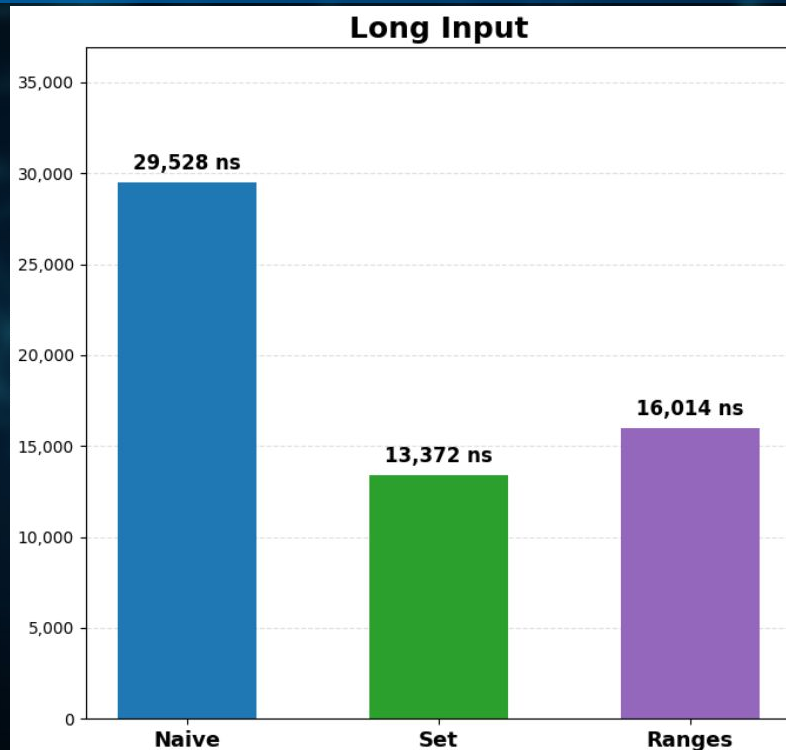
Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

Benchmarking The Simple Group Subtraction



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Benchmarking The Simple Group Subtraction



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Idea 1: Sort and Binary Search

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

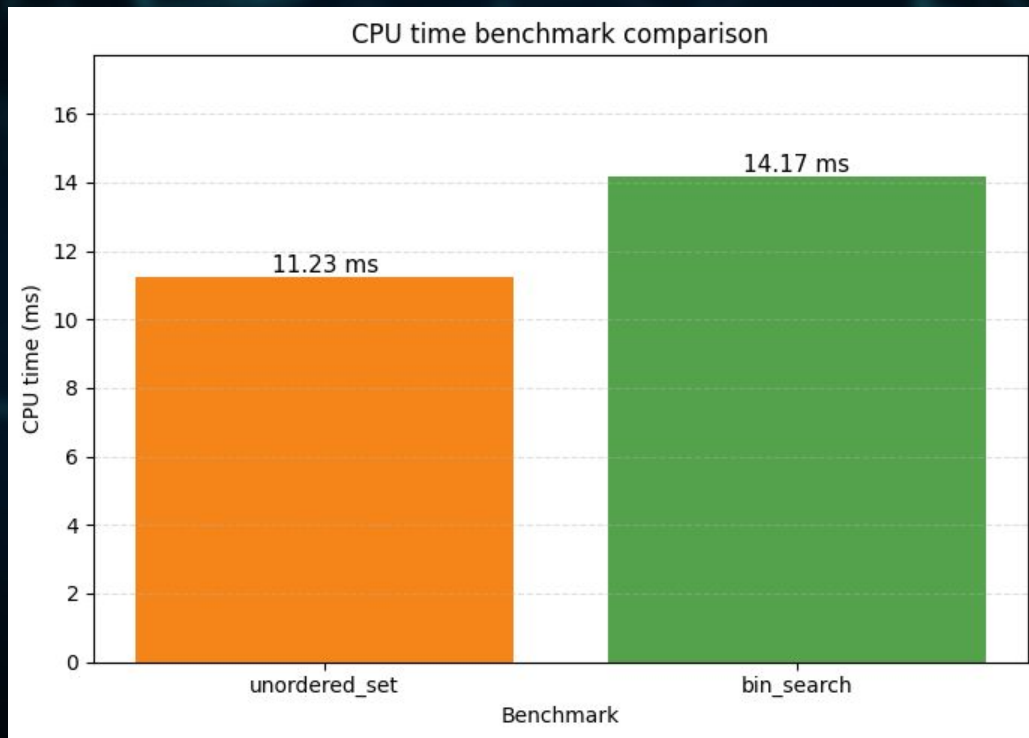
Benchmarking The Simple Group Subtraction

```
auto sorted_b = b;  
std::sort(sorted_b.begin(), sorted_b.end());  
sorted_b.erase(  
    std::unique(sorted_b.begin(), sorted_b.end()),  
    sorted_b.end());
```

Benchmarking The Simple Group Subtraction

```
for (auto value : a) {  
    if (!std::binary_search(b.begin(), b.end(), value)) {  
        result.push_back(value);  
    }  
}
```

Benchmarking The Simple Group Subtraction



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Idea 2: Dense Byte Search

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

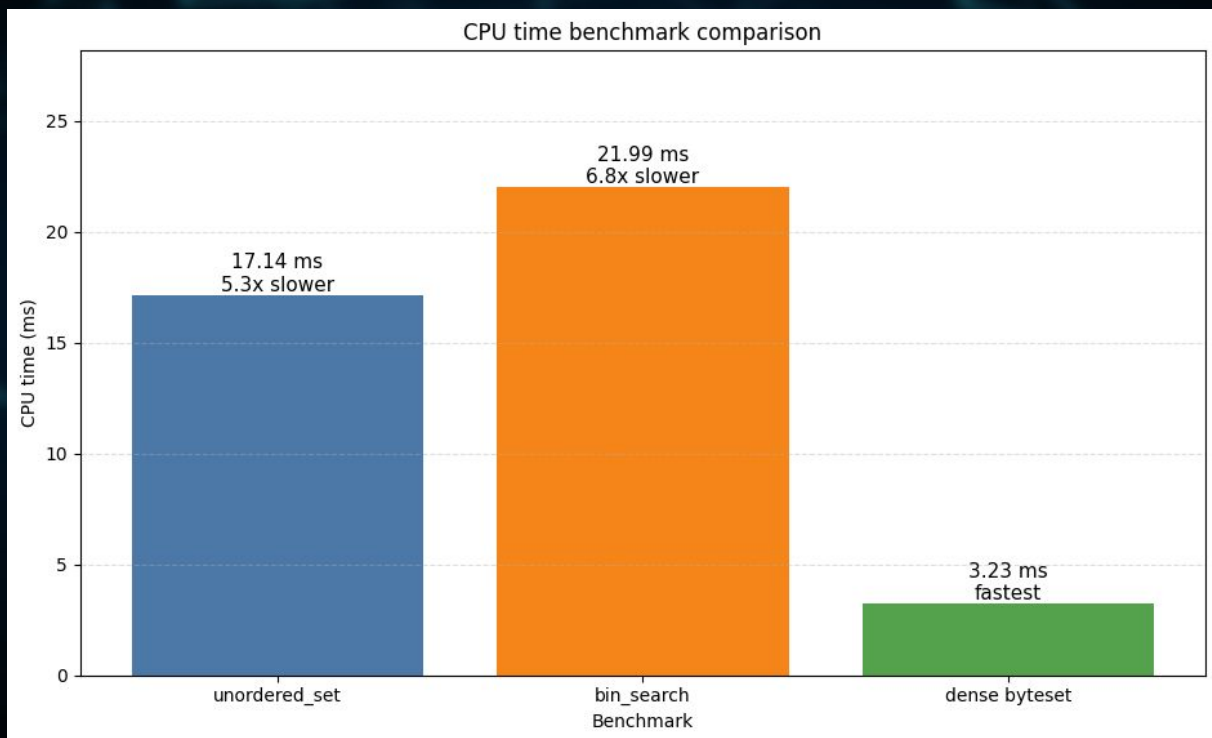
Benchmarking The Simple Group Subtraction

```
std::vector<std::uint8_t> values_to_remove(MAX_VALUE + 1, 0);  
  
for (auto value : b) {  
    values_to_remove[value] = 1;  
}
```

Benchmarking The Simple Group Subtraction

```
for (auto value : a) {  
    if (!values_to_remove[value]) {  
        result.push_back(value);  
    }  
}
```

Benchmarking The Simple Group Subtraction



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Idea 3: Bitmap

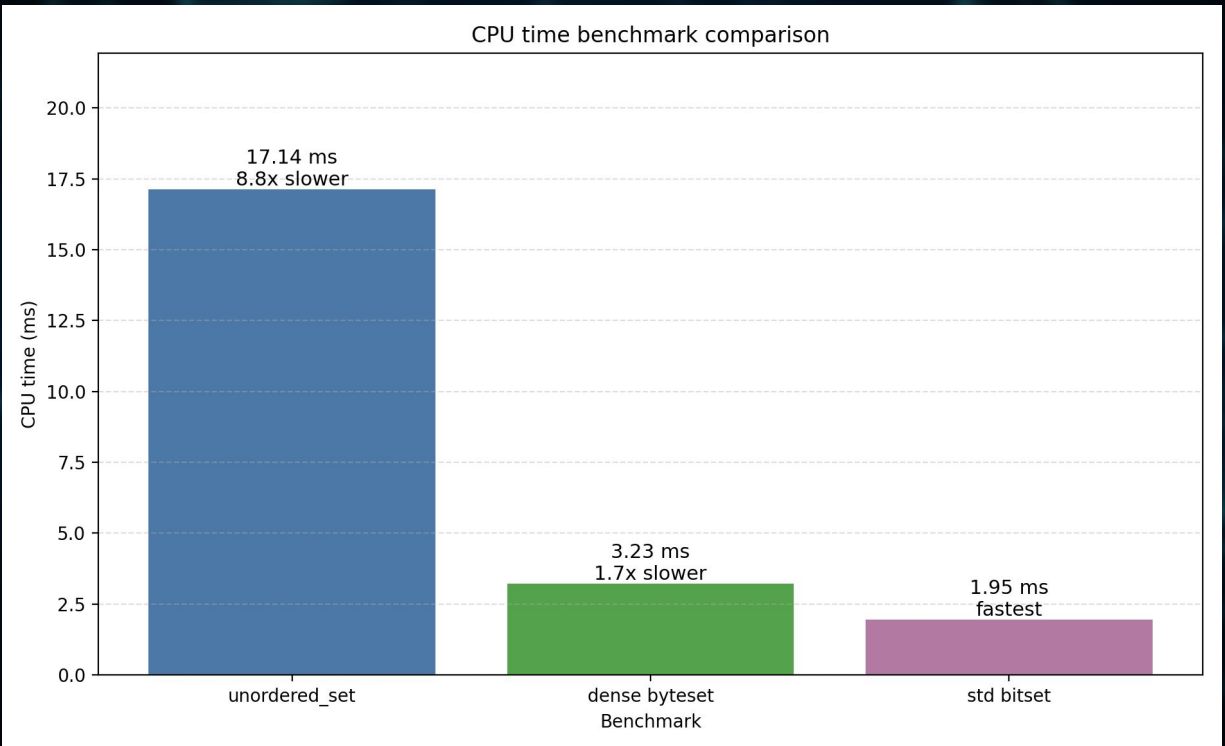
Benchmarking The Simple Group Subtraction

```
std::bitset<MAX_VALUE + 1> values_to_remove;  
for (auto value : b) {  
    values_to_remove.set(value);  
}
```

Benchmarking The Simple Group Subtraction

```
for (auto value : a) {  
    if (!values_to_remove.test(value)) {  
        result.push_back(value);  
    }  
}
```

Benchmarking The Simple Group Subtraction

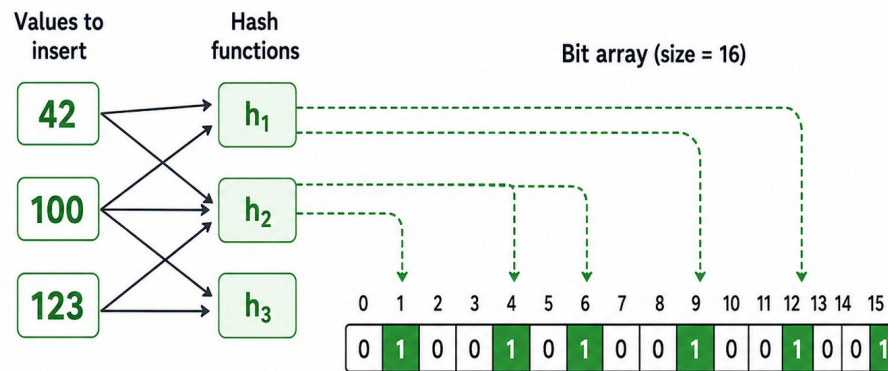


[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky



Idea 4: Bloom Filter

Bloom Filter: Insert



To insert a value, compute several hash functions and set the corresponding bits to 1.

Benchmarking The Simple Group Subtraction

```
void add(std::uint64_t value) {  
    const auto h1 = mix(value);  
    const auto h2 = mix(value ^ 0x9e3779b97f4a7c15ull);  
  
    for (std::size_t i = 0; i < BLOOM_HASH_COUNT; ++i) {  
        set_bit((h1 + i * h2) & BLOOM_MASK);  
    }  
}
```

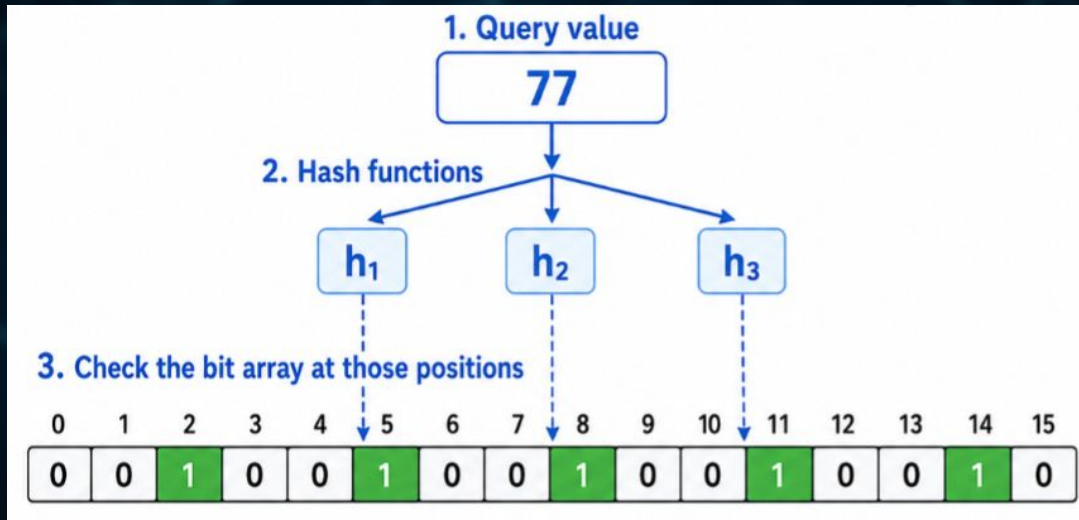
Benchmarking The Simple Group Subtraction

```
static std::uint64_t mix(std::uint64_t x) {  
    x ^= x >> 30;  
    x *= 0xbf58476d1ce4e5b9ull;  
    x ^= x >> 27;  
    x *= 0x94d049bb133111ebull;  
    x ^= x >> 31;  
    return x;  
}
```

Benchmarking The Simple Group Subtraction

```
void set_bit(std::uint64_t index) {  
    words_[index / 64] |= std::uint64_t{1} << (index % 64);  
}
```

Benchmarking The Simple Group Subtraction



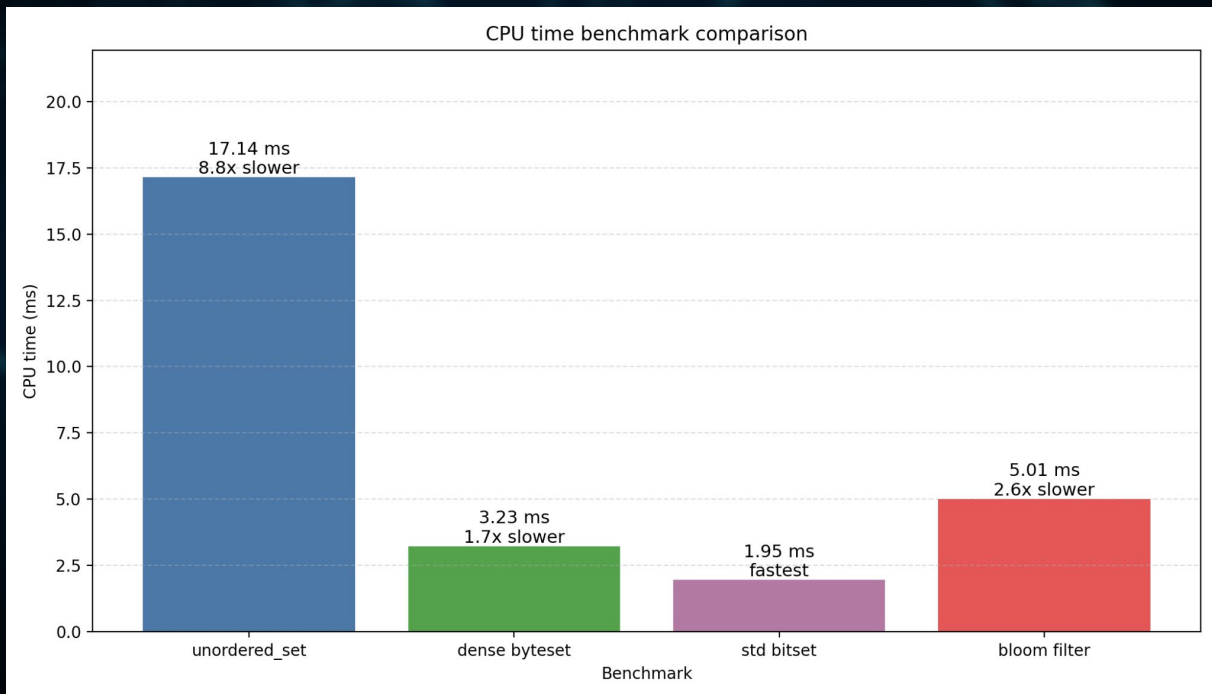
Benchmarking The Simple Group Subtraction

```
bool possibly_contains(std::uint64_t value) const {  
    const auto h1 = mix(value);  
    const auto h2 = mix(value ^ 0x9e3779b97f4a7c15ull);  
  
    for (std::size_t i = 0; i < BLOOM_HASH_COUNT; ++i) {  
        if (!test_bit((h1 + i * h2) & BLOOM_MASK)) {  
            return false;  
        }  
    }  
  
    return true;  
}
```

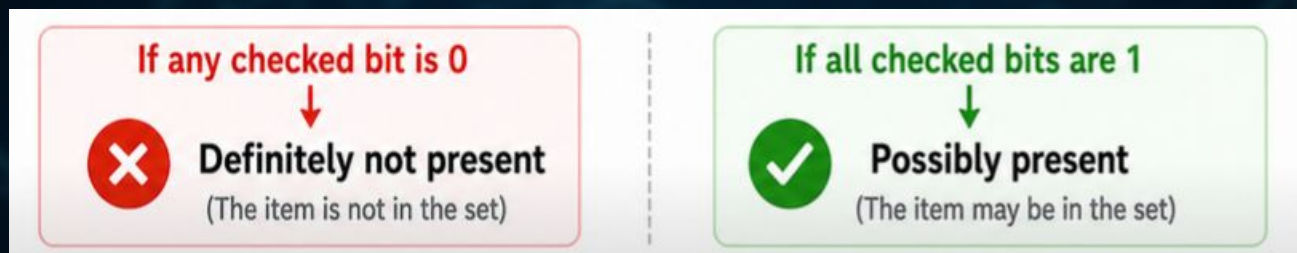
Benchmarking The Simple Group Subtraction

```
for (auto value : a) {  
    if (!bloom.possibly_contains(value)) {  
        result.push_back(value);  
    }  
}
```

Benchmarking The Simple Group Subtraction



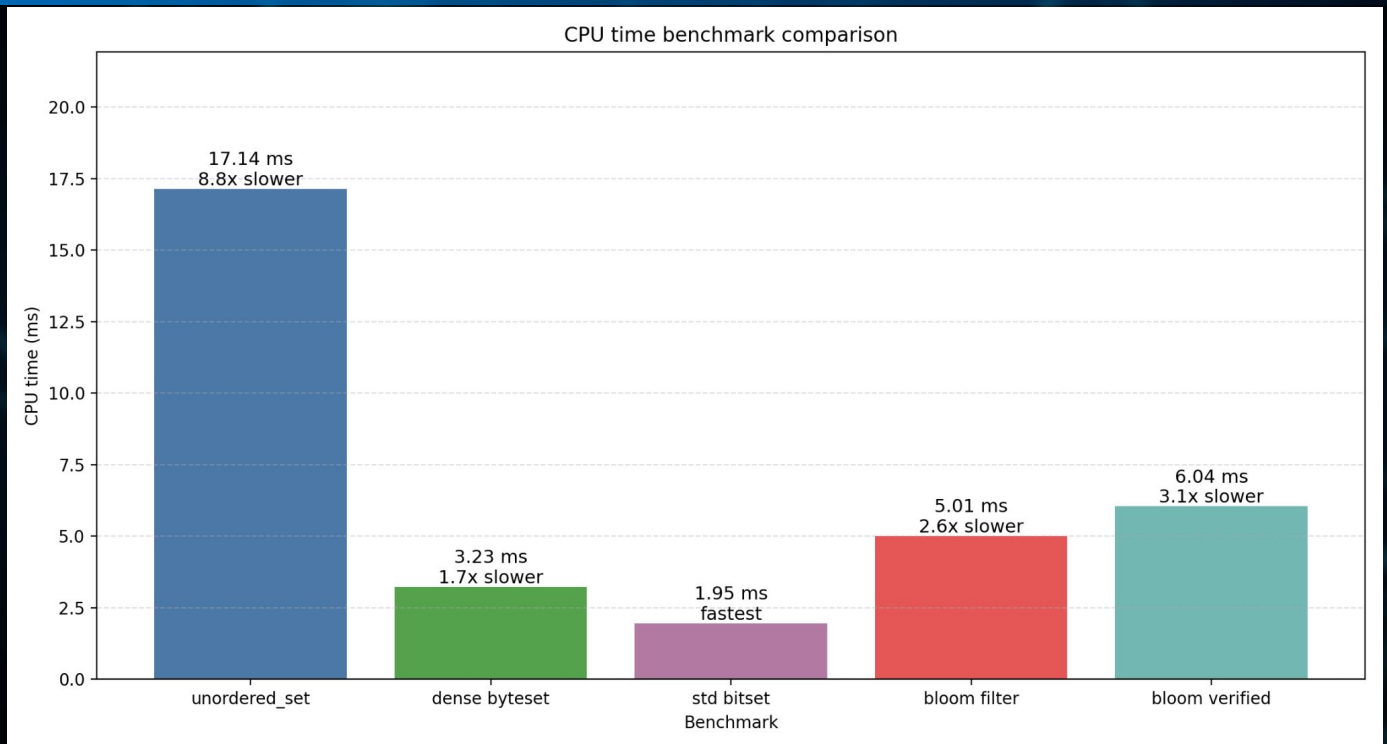
Benchmarking The Simple Group Subtraction



Benchmarking The Simple Group Subtraction

```
for (auto value : a) {  
    if (!bloom.possibly_contains(value)) {  
        result.push_back(value);  
        continue;  
    }  
  
    if (!std::binary_search(sorted_b.begin(), sorted_b.end(), value)) {  
        result.push_back(value);  
    }  
}
```

Benchmarking The Simple Group Subtraction



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Idea 5: Sorted Bucket Search

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

Benchmarking The Simple Group Subtraction

```
for (auto value : values) {  
    const auto bucket_index = value / BUCKET_RANGE;  
  
    if (bucket_index < BUCKET_COUNT) {  
        buckets[bucket_index].push_back(value);  
    }  
}
```

Benchmarking The Simple Group Subtraction

```
for (auto& bucket : buckets) {  
    std::sort(bucket.begin(), bucket.end());  
    bucket.erase(  
        std::unique(bucket.begin(), bucket.end()),  
        bucket.end()  
    );  
}
```

Benchmarking The Simple Group Subtraction

```
const auto bucket_index = value / BUCKET_RANGE;

if (bucket_index >= BUCKET_COUNT) {
    return false;
}

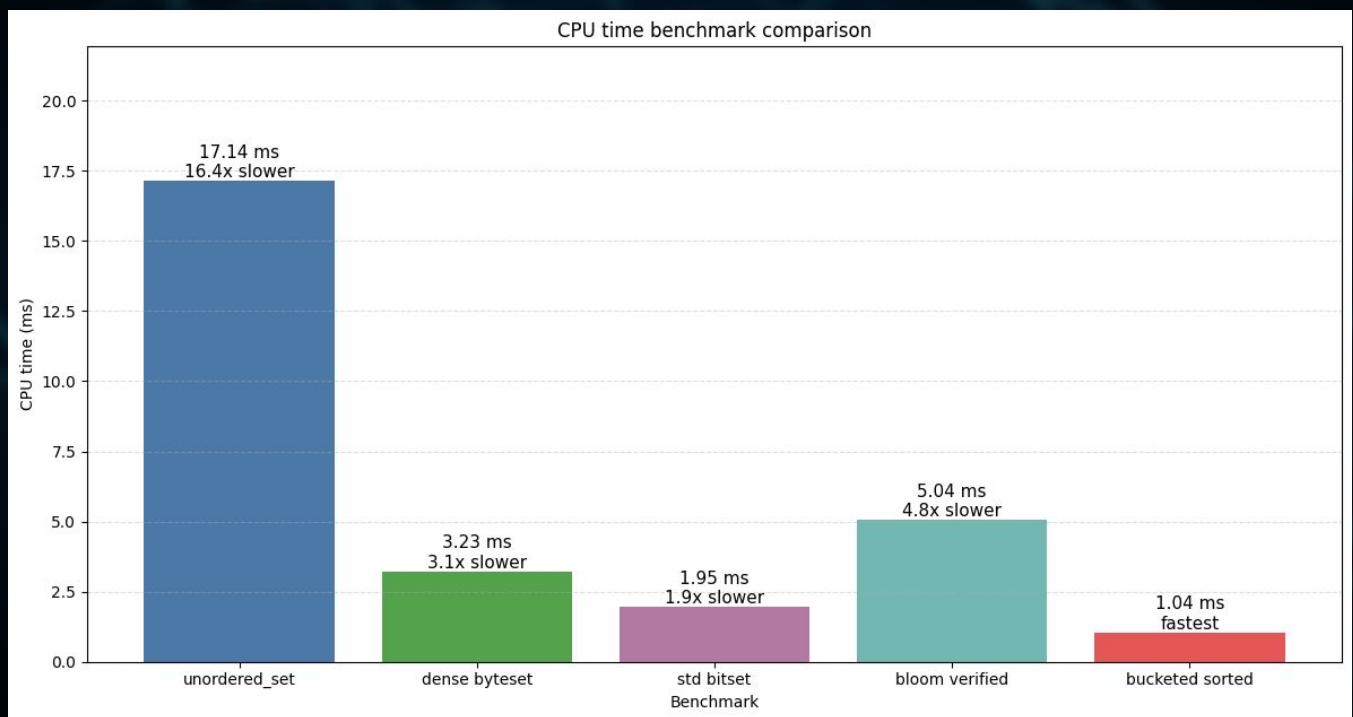
const auto& bucket = buckets[bucket_index];

return std::binary_search(
    bucket.begin(),
    bucket.end(),
    value
);
```

Benchmarking The Simple Group Subtraction

```
for (auto value : a) {  
    if (!lookup.contains(value)) {  
        result.push_back(value);  
    }  
}
```

Benchmarking The Simple Group Subtraction



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

Idea 6: Buckets of Byte Set Search

Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatakovsky

Benchmarking The Simple Group Subtraction

```
for (auto value : values) {  
    const auto bucket_index = value / BUCKET_RANGE;  
    const auto local_index = value % BUCKET_RANGE;  
  
    if (bucket_index < BUCKET_COUNT) {  
        buckets[bucket_index][local_index] = 1;  
    }  
}
```

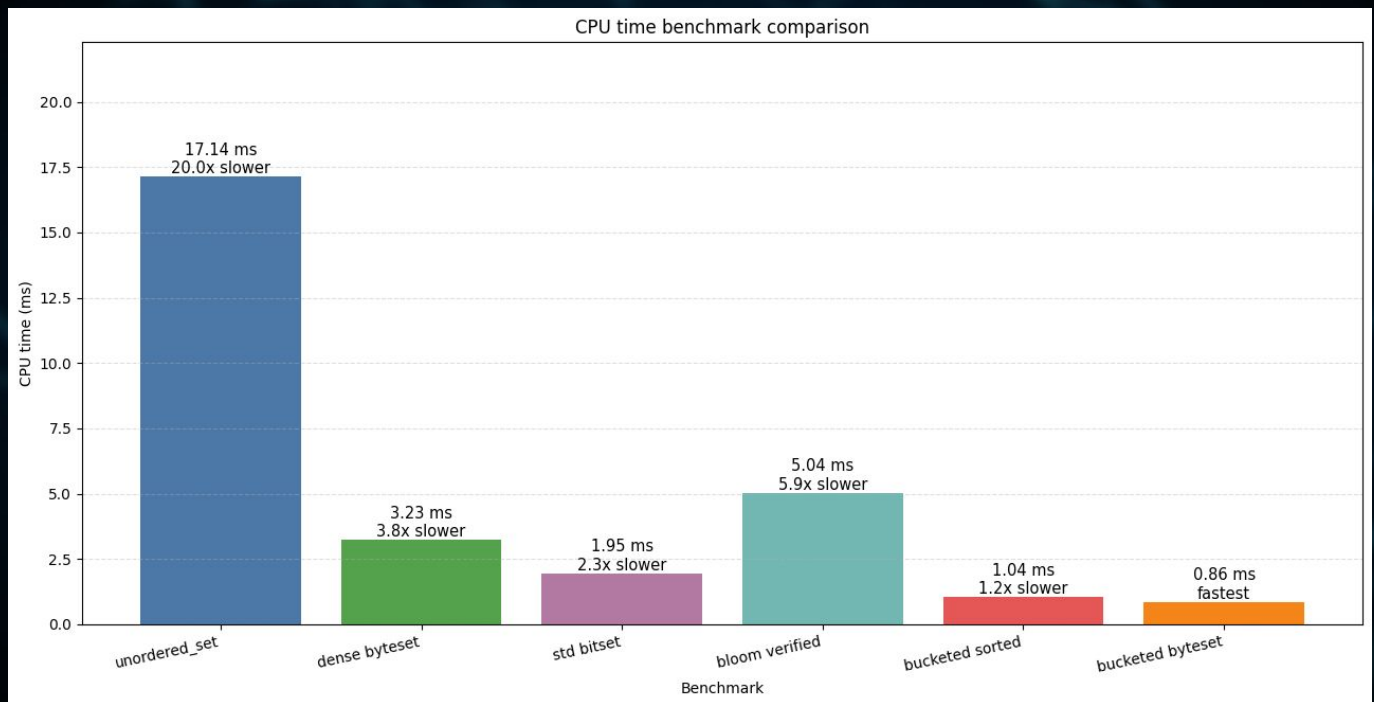
Benchmarking The Simple Group Subtraction

```
const auto bucket_index = value / BUCKET_RANGE;  
const auto local_index = value % BUCKET_RANGE;  
  
if (bucket_index >= BUCKET_COUNT) {  
    return false;  
}  
  
return buckets[bucket_index][local_index] == 1;
```

Benchmarking The Simple Group Subtraction

```
for (auto value : a) {  
    if (!lookup.contains(value)) {  
        result.push_back(value);  
    }  
}
```

Benchmarking The Simple Group Subtraction

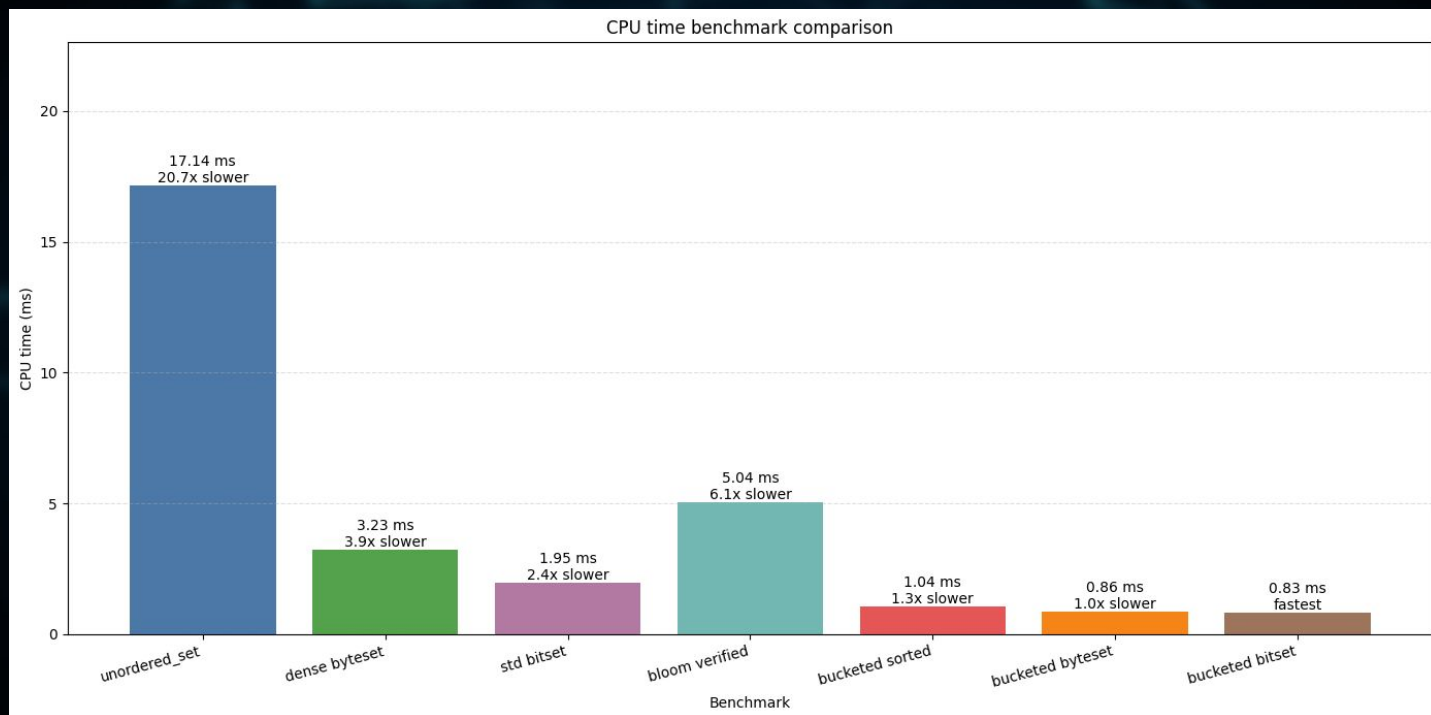


[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alex-dathskovsky

Idea 7: Buckets of Bit Set Search

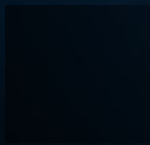
Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

Benchmarking The Simple Group Subtraction



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatkovsky

You Have The Power



[Alex Dathskovsky](#) | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

QUESTIONS



Alex Dathskovsky | alex.dathskovsky@scylladb.com | www.linkedin.com/in/alexdatahskovsky

THANK YOU FOR LISTENING

